Final Report: Alternate Semester

on

DEVELOPMENT OF A SMART IOT HEALTH MONITORING AND SENTRY ALERT SYSTEM

Submitted by
Avi Bansal
102105113
(B. E., Electronics (Instrumentation & Control) Engineering)

Under the Guidance of

Faculty Supervisor
Dr. Swati Sondhi
Associate Professor
DEIE, TIET, Patiala

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

2025

Department of Electrical and Instrumentation Engineering
Thapar Institute of Engineering and Technology, Patiala
(Declared as Deemed-to-be-University u/s 3 of the UGC Act., 1956)
Post Bag No. 32, Patiala – 147004
Punjab (India)

Declaration

I hereby declare that the final report titled, "Development of A Smart Iot Health Monitoring and Sentry Alert System", submitted as a partial requirement for the Alternate Semester, Design Project (UEI897) course towards the Bachelor of Engineering degree in Electronics (Instrumentation & Control) Engineering at Thapar Institute of Engineering and Technology, Patiala, is an accurate representation of the work I undertook. This work was carried out under the guidance and supervision of Dr. Swati Sondhi, Associate Professor at the Department of Electrical and Instrumentation Engineering, TIET, Patiala, India.

Place: Patiala **Avi Bansal**

Date: 15 July 2025 **102015113**

It is certified that the above statement made by the student is correct to the best of my knowledge and belief.

Faculty Supervisor
Dr. Swati Sondhi
Associate Professor
DEIE, TIET, Patiala

Acknowledgement

My sincere gratitude goes out to everyone who made it possible for me to do this assignment with their help. But without the kind support and help from multiple people and organizations, it would not be possible. I sincerely appreciate each and every one of them. I owe a debt of gratitude to Dr. Swati Sondhi for giving me the information I needed about what I do. Their unwavering assistance was crucial to finishing this Project report. The assistance and encouragement I received from my friend Samarth Yadav enabled me to do this assignment, and for that I am grateful.

Avi Bansal

Abstract

Sentry alert systems play a critical role in enhancing the operational efficiency and safety of military checkposts. Unlike conventional systems that rely solely on human vigilance, the proposed solution integrates wearable health monitoring, motion detection, and real-time alerting to provide a proactive, technology-driven approach to surveillance. This capstone project aims to develop an IoT-based sentry alert system capable of monitoring soldiers' vital health parameters and detecting unauthorized movements within a secured perimeter. The proposed system includes health monitoring nodes equipped with sensors that continuously track vital signs such as heart rate, body temperature, and fatigue levels. These nodes wirelessly transmit data using stable Wi-Fi protocols to a central server for evaluation. Simultaneously, motion detection nodes monitor the surrounding area for suspicious or unauthorized movement and immediately trigger local alerts using buzzers and indicators when such activity is detected.

All incoming data is processed by a centralized server node, which hosts a real-time dashboard displaying both health and motion-related information. This interface allows monitoring personnel to view live metrics, log critical incidents, and take necessary action swiftly. The dashboard also provides alert notifications when predefined thresholds are crossed, allowing prompt intervention and decision-making. By combining real-time biometric monitoring and motion surveillance into one cohesive system, this project enhances situational awareness, reduces manual workload, and improves response times. The integration of IoT technology and centralized monitoring in the proposed sentry alert system marks a significant step toward smarter, safer, and more efficient defense infrastructure.

Table of Contents

Declaration ii
Acknowledgement iii
Abstract iv
List of Figures viii
List of Tables ix
List of Abbreviations x

Chapter 1. Introduction
1.1 Introduction 3
1.2 Overview of the System 3
1.2.1 Sentry Wearable Node 3
1.2.2 Motion Detection Nodes 3
1.2.3 Dashboard Node and Interface 3
1.3 Communication Architecture 4
1.3.1 ESP-NOW Layer 4
1.3.2 HTTP and WebSocket Layer 4
1.4 Features of the System 4
1.4.1 Edge-Only Operation 4
1.4.2 Multi-Modal Alerting System 5
1.4.3 Modular and Scalable Design 5
1.5 Importance and Application Scope 5

Chapter 2. Literature Review
2.1 Literature Review 6
2.1.1 Wearable Health Monitoring in Defense 6
2.1.2 Motion Detection Systems for Sentry Surveillance 7
2.1.3 IoT-Based Real-Time Monitoring Dashboards 8
2.1.4 Limitations and Emerging Trends 8
2.2 Statement of Problem-Based on Identified Research Gaps 9

Chapter 3. Hardware Architecture and Communication Design
3.1 Overview 10
3.2 Node-Level Architecture 10
3.3 Hardware Components Used 11
3.4 Sentry Watch Node 17

3.5 Motion Detection Nodes 17
3.6 Control Dashboard Node (Sentry Panel) 17
3.7 Communication Flow 20
3.8 Power and Packaging 20
3.9 Circuit Schematic Description of Control Node 21
3.10 Circuit Schematic Description of Motion Node 22

Chapter 4. Prototype Deployment and Results
4.1 Implementation Overview 23
4.2 Hardware Deployment Results 23
4.3 Motion Detection Testing 25
4.4 Vital Monitoring Testing 27
4.5 Result Analysis 29
4.6 Discussions and Inferences Drawn 30
4.7 Justification of Objectives Achieved 30

Chapter 5. Conclusions and Future Work
5.1 Conclusions 31
5.2 Future Work 31

References 33
Annexure
A.1 CPP PROGRAM CODE FOR MOTION NODE 37
A.2 CPP PROGRAM CODE FOR SENTRY PANEL 40
A.3 CPP PROGRAM CODE FOR WATCH NODE 44
A.4 CPP PROGRAM CODE FOR SERVER NODE 47

Plagiarism Report 62

List of Figures

Figure No.	Figure Caption	Page No.
1.1	Block Diagram of Sentry Alert and Monitoring System	2
3.1	ESP 8266 D1 Mini	12
3.2	Accelerometer & Gyroscope Sensor	12
3.3	Heart Rate and SpO_2 Sensor	13
3.4	Buck Converter Mini 360	13
3.5	PIR Motion Sensor	13
3.6	OLED Display	13
3.7	Buzzer	14
3.8	Battery	14
3.9	JST Connector 2-pin	15
3.10	12V 2A SMPS Adapter	15
3.11	Flow diagram of Health Node	18
3.12	Flow Diagram of Motion Node	19
3.13	Flow Diagram of Control Node	20
3.14	Schematic of Control Node	21
3.15	Schematic of Motion Node	22
4.1	Prototype IOT System for Motion Node	24
4.2	Prototype IOT System for Watch Node	24
4.3	Prototype IOT System for Sentry Panel (Control Node)	25
4.4	Server Dashboard indicating Motion Detected	26
4.5	Server Dashboard indicating NO Motion	26
4.6	Motion Alert Display on OLED	27
4.7	Server Dashboard indicating Awake Status	28
4.8	Server Dashboard indicating Fainted Status	29
4.9	Server Dashboard indicating Moving Status	29

List of Tables

Table No.	Table Caption	Page No.
3.1	Summary of Hardware Components used	16

List of Abbreviations

IoT	Internet of Things
Wi-Fi	Wireless Fidelity
HTTP	HyperText Transfer Protocol
OLED	Organic Light Emitting Diode
ESP	Espressif (family of microcontrollers)
ESP8266 / ESP32	Espressif Systems Wi-Fi SoC models
MPU6050	Motion Processing Unit 6050 (Accelerometer + Gyroscope)
MAX30102	Pulse Oximeter and Heart-Rate Sensor module
PIR	Passive Infrared (Sensor)
LED	Light Emitting Diode
I^2C	Inter-Integrated Circuit
ADC	Analog-to-Digital Converter
GUI	Graphical User Interface
PCB	Printed Circuit Board
SpO_2	Peripheral Capillary Oxygen Saturation
BPM	Beats Per Minute
MAC	Media Access Control (Address)
UDP	User Datagram Protocol
URL	Uniform Resource Locator
API	Application Programming Interface
QoS	Quality of Service
PWM	Pulse Width Modulatio

Chapter 1
Introduction

Chapter Outline

This chapter introduces the concept of IoT-based monitoring for sentry security systems and outlines the key challenges faced in remote or critical locations. It explains the role of wearable sensors and surrounding motion detection nodes, as well as the architecture of the system, communication protocols used, and the advantages of operating entirely on edge devices. The chapter concludes with a breakdown of the key features of the system and the motivations behind its development.

1.1 Introduction

Traditional security and surveillance systems, particularly in high-risk zones like military outposts and critical infrastructure facilities, have long relied on manual monitoring, static camera networks, and reactive alert mechanisms. While these methods have served their purpose for decades, they often suffer from delayed responses, oversight due to human fatigue, and the inability to detect subtle signs of distress—especially among on-duty personnel such as sentries. In such high-stakes environments, even a few seconds of delayed action or unrecognized warning signs can result in serious breaches or loss of life [1].

Conventional systems also tend to overlook the human aspect of security—specifically, the health and situational awareness of the personnel involved. Soldiers and security guards are often required to endure prolonged hours of vigilance under extreme environmental and psychological stress, without any support systems in place to monitor their well-being or fatigue levels. This presents a significant vulnerability in both safety and operational effectiveness [2].

In response to these limitations, the integration of modern IoT-based technologies offers a promising solution. Low-power microcontrollers, wearable health sensors, and intelligent motion detectors can now be combined to form real-time, autonomous monitoring systems. [3]. These systems not only monitor physiological parameters such as heart rate and movement but also enable immediate alerts in case of abnormal patterns—whether related to health deterioration or perimeter intrusions [4]. When paired with a centralized dashboard for supervisors, such systems allow data-driven decision-making and rapid intervention in critical

moments [5]. By leveraging this approach, the project proposes a forward-thinking model for improving situational awareness and personal safety in surveillance scenarios. It reflects a shift from passive observation to active, intelligent monitoring—bridging the gap between personnel wellness and security threat detection in an integrated, real-time ecosystem [6].

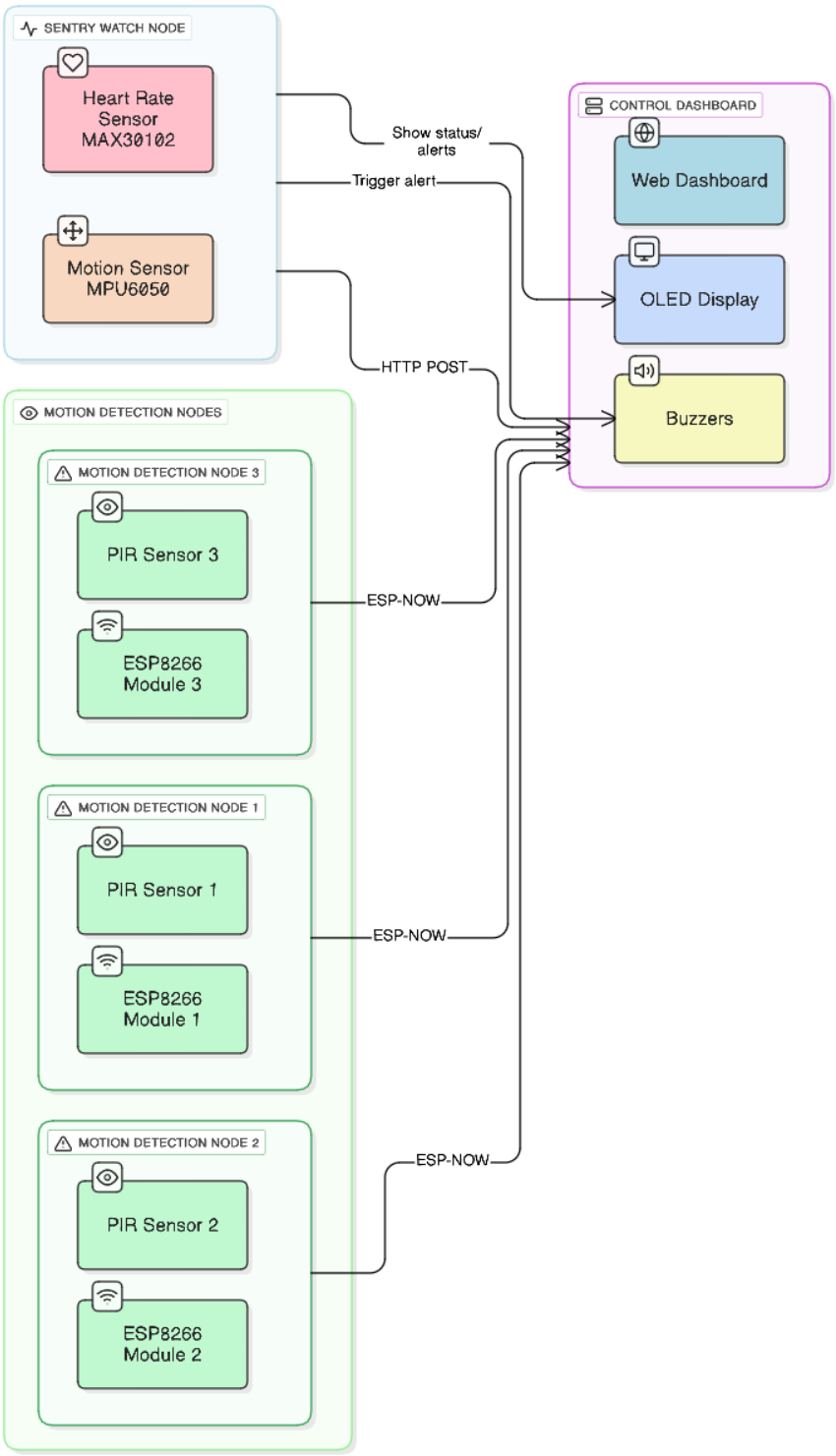

Fig 1.1 – Block Diagram of Sentry Alert and Monitoring System

1.2 Overview of the System

The proposed Sentry Alert System adopts a distributed IoT network architecture, enabling cooperative interaction between multiple sensor units. It consists of three core functional blocks as shown in Fig 1.1, the sentry's wearable monitoring node, multiple static motion detection nodes placed around the perimeter, and a central dashboard node that aggregates, displays, and responds to the data. Each component works in real time and is optimized for reliability, low power consumption, and offline operation [7].

1.2.1 Sentry Wearable Node

The wearable unit is equipped with:

- MAX30102 sensor for heart rate monitoring.
- MPU6050 accelerometer-gyroscope module for motion tracking.
- An OLED screen for live display of vital signs.
- Dual buzzers to raise alarms during emergency scenarios.

This node continuously tracks the user's movement and pulse rate. It determines if the sentry is standing still for too long or has fainted, and accordingly activates alarms or sends alerts to the central dashboard [8].

1.2.2 Motion Detection Nodes

The system includes three strategically placed PIR-based motion detection nodes, each fitted with an HC-SR501 sensor. Only one node remains active at any time to prevent interference and reduce power consumption. These nodes:

- Operate using ESP-NOW protocol to send data directly to the dashboard node [9].
- Detect suspicious activity in the vicinity and alert the control center instantly.
- Operate independently of Wi-Fi, improving stability and security.

1.2.3 Dashboard Node and Interface

The dashboard node hosts a local web server built using ESPAsyncWebServer and WebSockets. It offers:

- Real-time display of the sentry's health and activity status (e.g., "Awake", "Fainted", "Disconnected").
- Visual alerts for motion detection and physiological anomalies.
- No need for internet access or external databases [10].

1.3 Communication Architecture

The system's data exchange is divided into two efficient and lightweight communication layers to ensure fast, seamless interaction between nodes. These layers are optimized to reduce latency, save power, and maintain operability even in disconnected environments.

1.3.1 ESP-NOW Layer

This layer manages the communication from motion detection nodes to the dashboard node. It uses a low-power protocol designed for quick, peer-to-peer interaction without Wi-Fi connectivity [11].

- Used for motion detection nodes to communicate directly with the dashboard.
- Enables low-power, connectionless communication between ESP8266 devices.
- Ensures instant alerts without the need for a Wi-Fi network or router.

1.3.2 HTTP and WebSocket Layer

This layer handles the transmission of biometric and motion data from the wearable node to the dashboard. HTTP provides structured periodic updates, while WebSocket enables real-time, push-based interaction.

- Used by the sentry wearable node to transmit vital signs data to the dashboard.
- HTTP is used for periodic updates, while WebSockets allow real-time, bi-directional communication.
- The dashboard updates dynamically without requiring page reloads [12].

1.4 Features of the System

The system integrates hardware and software in a lightweight, edge-operated IoT architecture tailored for reliability and modularity. It achieves real-time situational awareness, combining internal and external monitoring while preserving operational independence from the cloud.

1.4.1 Edge-Only Operation

The system is designed to function entirely on local Wi-Fi or ESP-NOW without relying on cloud services or internet access. This ensures rapid communication, data privacy, and full offline functionality—ideal for remote or high-security environments [13].

- Entire system functions on local Wi-Fi or ESP-NOW.
- No cloud platforms, third-party APIs, or internet access required.
- Ensures offline functionality and fast response.

1.4.2 Multi-Modal Alerting System

Alerts are generated based on abnormal vitals, fainting, or prolonged inactivity, with feedback delivered both visually on the dashboard and audibly via buzzers. This ensures timely intervention through integrated health and motion-based triggers [14].

- Alerts for inactivity, fainting, or abnormal vitals.
- Visual dashboard cues and audible alarms via buzzers.
- Combines wearable and perimeter alerts into a single monitoring pipeline.

1.4.3 Modular and Scalable Design

The architecture supports easy expansion by adding or removing nodes without disrupting system functionality. It uses cost-effective, open-source components and a flexible UI that scales to monitor multiple personnel or zones [15].

- Supports easy addition or removal of motion nodes.
- Flexible dashboard interface can accommodate multiple users or sentries.
- Components are low-cost and open-source, promoting scalability.

1.5 Importance and Application Scope

This system provides a comprehensive and dependable solution for real-time monitoring of both personnel and surroundings. It is especially useful in isolated and high-risk environments such as border outposts, disaster zones, military checkpoints, and large-scale construction sites [16]. By operating independently of external networks, it ensures continuous monitoring even during communication blackouts or internet outages [17].

The system enhances both situational awareness and personal safety by responding promptly to critical events such as health deterioration, fainting, or unauthorized motion. Its offline-first design ensures sensitive data remains local, promoting both privacy and faster emergency response. These qualities make it ideal for adoption in government, defense, and emergency operations where lives and security depend on uninterrupted, intelligent monitoring [18].

Chapter 2
Literature Review

Chapter Outline

This chapter presents a comprehensive review of existing literature in the areas of wearable health monitoring, motion detection systems, and IoT-based real-time dashboards for defense and surveillance applications. The discussion begins with advancements in wearable technologies for monitoring soldier health, followed by innovations in motion sensing for perimeter security. The chapter also explores the role of real-time IoT dashboards in centralized monitoring and decision-making. The concluding section identifies key technological gaps and suggests directions for future research based on current limitations in system integration, accuracy, and reliability.

2.1 Literature Review

Recent advancements in defense technologies have placed increased emphasis on integrating wearable health sensors, smart surveillance mechanisms, and IoT-based data processing systems. These developments aim to enhance situational awareness, operational safety, and decision-making capabilities in military environments, particularly at critical sentry posts and field operations.

2.1.1 Wearable Health Monitoring in Defense

Wearable technologies play a crucial role in modern military systems by enabling continuous monitoring of a soldier's physiological parameters such as heart rate, temperature, stress, and fatigue levels. These real-time metrics provide invaluable insight into the physical condition of personnel deployed in high-risk or remote locations.

a) Importance of Continuous Health Tracking: According to Pantelopoulos and Bourbakis, wearable sensor-based systems offer proactive health surveillance that supports early diagnosis, risk mitigation, and performance optimization in defense scenarios [19]. By ensuring continuous feedback on vital signs, such systems help commanders make informed decisions regarding soldier readiness and medical intervention.

b) Applications in High-Stress Military Environments: Majumder et al. emphasize the significance of such technologies in high-stress settings, such as border checkposts or combat zones, where real-time data from wearable sensors can be used to detect symptoms

of fatigue, heatstroke, or cardiovascular strain [20]. These insights contribute to reducing battlefield casualties through early detection and prevention.

c) Technological Innovations and Sensor Miniaturization: The improvement in wearable systems stems from innovations in miniaturized, low-power biosensors and wireless communication. Lee and Suh have shown that inertial sensor systems designed specifically for soldiers can enhance positioning accuracy without compromising mobility [21]. These improvements allow wearable devices to operate seamlessly, even in dynamic field environments.

d) Challenges in Deployment: Despite these advancements, several limitations remain. Sensor calibration errors, environmental interference, and poor sensor placement can affect data integrity. Gubbi et al. further point out that ensuring consistent wireless transmission and power management is critical for real-time monitoring systems, especially in remote or signal-restricted areas [22].

2.1.2 Motion Detection Systems for Sentry Surveillance

In addition to health monitoring, surveillance systems rely on robust motion detection technologies to identify unauthorized movements around protected zones, such as army checkpoints or restricted perimeters.

a) Motion Recognition through Image Processing: Zhao and Cheung discuss spatial-temporal feature extraction methods such as Motion Energy Images (MEI), which are effective in identifying and classifying human activities under varying environmental conditions [23]. These methods form the core of automated motion detection systems used to distinguish between civilian and potentially hostile intrusions.

b) Reducing False Alarms and Improving Accuracy: To minimize false alerts and improve reliability, machine learning techniques have been employed for motion classification. Singh et al. highlight the use of multiple classifiers, including SVM and KNN, in intrusion detection systems to refine decision-making and reduce noise in sensor inputs [24].

c) Real-Time Alert Generation: By integrating motion detection mechanisms with real-time alert systems, unauthorized activity can be reported to the control center instantly. This integration improves situational responsiveness, allowing soldiers to act swiftly. Such systems have become essential in modern military infrastructures where surveillance coverage is required 24/7.

2.1.3 IoT-Based Real-Time Monitoring Dashboards

IoT technology plays a pivotal role in aggregating and visualizing data from distributed sensor nodes, thereby enabling centralized and remote monitoring of both health metrics and motion events.

a) Architecture and Data Aggregation: Ali et al. proposed an IoT-based smart health monitoring system that leverages wireless sensor networks to transmit data to centralized dashboards, supporting real-time diagnosis and monitoring [25]. Similarly, Paxson's Bro system offers real-time event analysis and anomaly detection through a distributed sensor network, proving its utility in security and monitoring domains [26].

b) Scalability and Interoperability in Defense Zones: Yick et al. underline the need for scalable and energy-efficient wireless sensor networks in large-scale defense deployments [27]. Their survey suggests that intelligent network design, including routing protocols and fault tolerance, is key to achieving high reliability and low latency in mission-critical settings.

c) Human-Cloud Integration: Chen et al. introduced the concept of Wearable 2.0, emphasizing the shift from basic sensing to integrated human-cloud health management systems. These systems allow seamless syncing of soldier vitals with remote dashboards, enabling healthcare professionals and commanders to access up-to-date health records from anywhere [28].

d) Visual Dashboards for Real-Time Decision Support: Dashboards that visualize real-time sensor data have been reviewed by Kumar and Patel, who discuss how graphical interfaces aid in interpreting complex patterns and identifying urgent anomalies [29]. This functionality is essential for defense operators managing large-scale surveillance and health-monitoring networks.

2.1.4 Limitations and Emerging Trends

Despite the evident benefits of integrating wearable sensors, motion detection, and IoT dashboards, several limitations restrict the full deployment and adoption of such systems in military contexts.

a) Technical and Environmental Constraints: Al-Karaki and Kamal identify network latency, energy constraints, and dynamic topology changes as critical challenges in wireless military systems [30]. These factors can lead to intermittent communication or sensor failure, especially in remote locations with harsh environmental conditions.

b) Need for Robust, Integrated Systems: M. S. and Muhammad emphasize the importance of cloud-assisted industrial IoT (IIoT) frameworks in bridging the gap between isolated

sensor nodes and central processing units [31]. Such integrated architectures are necessary for large-scale deployments where both motion and physiological data must be interpreted concurrently.

c) Toward Next-Generation Wearables: Recent advances in soldier monitoring platforms include sensor fusion, edge computing, and machine learning for anomaly detection. Bhatia et al. describe the potential of next-generation wearable technologies in simultaneously improving health diagnostics and enhancing situational awareness through predictive algorithms and decentralized computing [32].

2.2 Statement of Problem Based on Identified Research Gaps

Although substantial progress has been made in the development of wearable health monitors, motion detection systems, and real-time IoT dashboards for military applications, several research gaps persist. Integration remains a core challenge: most existing systems focus on either health or security, but few offer a unified platform capable of simultaneously monitoring and reacting to physiological and environmental data. Moreover, unresolved issues such as signal interference, high power consumption, inconsistent data accuracy, and lack of protocol standardization hinder operational reliability. The literature also reveals limited studies on long-term performance and adaptability of wearable systems in harsh climatic or tactical environments. To address these gaps, future research must focus on:

- Developing integrated, lightweight, and ruggedized multi-sensor platforms.
- Enhancing wireless communication protocols for high-reliability and low-latency transmission.
- Leveraging edge-AI for local data processing to reduce network dependency.
- Improving human factors such as comfort, usability, and battery life in wearable systems.

Bridging these gaps will be essential for realizing a fully functional, real-time, and intelligent sentry alert and monitoring system tailored to modern defense needs.

Chapter 3
Hardware Architecture and Communication Design

Chapter Outline

This chapter describes the hardware and communication design of the system. It presents the key components of the Sentry Watch Node, Motion Detection Nodes, and Control Dashboard Node. Block diagrams and schematic placeholders illustrate the functional layout. Communication strategies, including ESP-NOW and HTTP/WebSocket protocols, are also explained. Packaging, power regulation, and overall deployment setup are addressed.

.

3.1 Overview

The proposed IoT-based Sentry Alert and Monitoring System is built around a multi-node architecture, each designed for specific roles in health monitoring, motion detection, and centralized alerting. The physical system comprises a wearable Sentry Watch Node, multiple Motion Detection Nodes, and a Control Dashboard Node that serves as the central monitoring interface.

These nodes are implemented using low-power ESP8266 microcontrollers and communicate through a hybrid protocol setup involving ESP-NOW and HTTP/WebSocket for fast, local, and offline-capable transmission. The hardware setup emphasizes modularity, compact packaging, and reliable power regulation through buck converters and Li-ion battery integration.

This chapter details the design and interconnection of each subsystem, communication logic, schematic interpretations, and a dedicated subsection for component specifications, offering a comprehensive view of how the system is realized in hardware.

3.2 Node-Level Architecture

The system is logically divided into three types of nodes, each fulfilling a distinct role in the surveillance and alert framework:

- Sentry Watch Node: A wearable device designed for real-time monitoring of a sentry's physiological conditions such as heart rate, SpO_2 levels, and posture. It also generates local alerts in case of abnormal health status or fainting events.

- Motion Detection Nodes: Three standalone units using PIR sensors to detect movement across predefined zones. Each node operates independently and communicates motion alerts wirelessly to the control dashboard.
- Control Dashboard Node: Acts as the central interface that receives data from the watch and motion nodes. It displays real-time statuses on an OLED display and triggers audible alerts when abnormal events are detected.

These nodes are powered using a combination of 3.7V Li-ion batteries, Mini 360 buck converters, and LDO regulators to ensure safe voltage levels. All microcontrollers are based on the ESP8266 D1 Mini, chosen for its compact size, onboard Wi-Fi capabilities, and compatibility with various sensors and displays.

3.3 Hardware Components Used

The system architecture leverages compact, low-power, and widely available IoT hardware components to maintain reliability, scalability, and cost-efficiency. These components are strategically distributed across three primary subsystems—Sentry Watch Node, Motion Detection Nodes, and the Control Dashboard Node. Each component was selected based on its operational stability, power requirements, sensing capabilities, and compatibility with ESP8266 microcontrollers.

ESP8266 D1 Mini Wi-Fi Module:

The D1 Mini, shown in Fig 3.1, is a compact development board based on the ESP-12F (ESP8266) chip. It supports Wi-Fi and provides multiple GPIOs for interfacing sensors. It is used to read sensor data (BPM, motion, etc.) and send it to the dashboard via HTTP and WebSocket protocols. Its small size and low power make it ideal for wearable and IoT applications [33].

Specifications:

- Microcontroller: Tensilica L106 32-bit
- Operating Voltage: 3.0–3.6V
- Flash Memory: 4MB
- GPIO Pins: 11 (usable)
- Wi-Fi Standard: IEEE 802.11 b/g/n
- Deep Sleep Current: ~20 µA
- Dimensions: ~34 x 25 mm

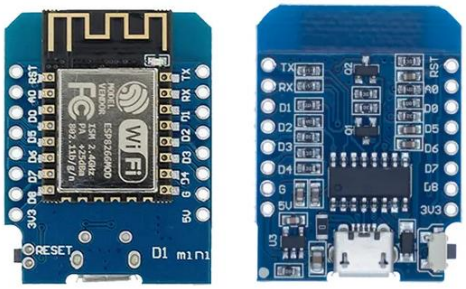

Fig 3.1 ESP8266 D1 Mini Wi-Fi Module

Fig 3.2 Accelerometer & Gyroscope Sensor

Accelerometer & Gyroscope Sensor MPU6050:

The MPU6050, shown in Fig 3.2, combines a 3-axis accelerometer and 3-axis gyroscope. It detects user movement, orientation, and fall/fainting events. In this project, it helps determine whether the person is "Moving", "Lying Still", or has "Fainted", based on acceleration and tilt thresholds [34].

Specifications:

- Supply Voltage: 3.3–5V
- Communication Interface: I2C
- Gyroscope Range: ±250 to ±2000 °/s
- Accelerometer Range: ±2g to ±16g
- Built-in DMP (Digital Motion Processor): Yes
- Module Size: 20 x 15 mm (approx.)

Heart Rate and SpO_2 Sensor MAX30102:

The MAX30102, shown in Fig 3.3, measures heart rate and oxygen saturation using red and infrared LEDs. It sends real-time BPM data to the dashboard. In this project, the sensor is attached to the finger or wrist to continuously monitor vitals, helping identify abnormal heart conditions or user activity status [35].

Specifications:

- Supply Voltage: 1.8V (Core), 3.3V (I/O)
- Interface: I2C
- LED Wavelengths: Red (660 nm), IR (880 nm)
- Active Mode Current: <600 µA
- Sleep Mode Current: 0.7 µA
- Size: ~14 x 14 mm

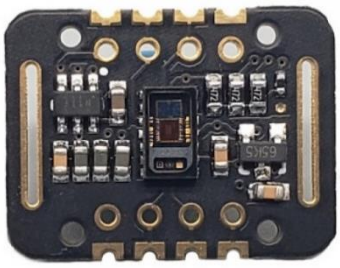

Fig 3.3 Heart Rate and SpO_2 Sensor

Fig 3.4 Buck Converter Mini 360

Buck Converter Mini 360:

The Mini 360, shown in Fig 3.4, is a small step-down voltage regulator. It takes the 3.7V–4.2V from a Li-ion battery and converts it to a stable 3.3V, which is needed to run the ESP8266 and sensors. Its high efficiency ensures minimal heat loss, making it suitable for wearable battery-powered projects [36].

Specifications:

- Input Voltage: 4.75V–23V
- Output Voltage: 1.0V–17V (adjustable)
- Max Output Current: 1.8A
- Efficiency: Up to 96%
- Switching Frequency: 340 kHz
- Module Size: 17 x 11 mm (approx.)

Fig 3.5 PIR Motion Sensor

Fig 3.6 OLED Display

PIR Motion Sensor HC-SR501:

The HC-SR501, shown in Fig 3.5, detects movement by sensing infrared radiation from a human body. It is used in standalone motion nodes (Node 1, 2, 3) to detect human presence. Each motion event is sent to the dashboard in real time for monitoring or alerting [37].

Specifications:

- Operating Voltage: 5V
- Detection Range: 3–7 meters
- Sensing Angle: 120°
- Delay Time: Adjustable (0.3s to 300s)
- Output: HIGH (3.3V) on motion detection
- Size: ~32 x 24 mm (excluding dome)

OLED Display (0.96" I2C):

The OLED Display, shown in Fig 3.6, is used to show the system's current status such as BPM, motion state, or connection alerts. It communicates with the ESP8266 via I2C and has low power usage. The screen provides clear text even in low light, making it ideal for real-time feedback in portable devices [38].

Specifications:

- Display Type: Monochrome (White/Blue)
- Resolution: 128 x 64 pixels
- Interface: I2C
- Supply Voltage: 3.3V–5V
- Power Consumption: ~20 mA
- Size: ~27 x 27 mm

Fig 3.7 Buzzer

Fig 3.8 Battery

Buzzer:

The buzzer, shown in Fig 3.7, is used to produce sound alerts when abnormal states are detected (like “Fainted” or sudden motion). Being an active buzzer, it does not require PWM; simply sending it a HIGH signal is enough to make it beep. This makes it easy to control from the ESP8266 [39].

Specifications:

- Operating Voltage: 3.3V–5V
- Sound Output: 85 dB (typical)
- Type: Active (continuous tone without PWM)
- Frequency: ~2 kHz
- Size: ~12 x 8 mm

Battery:

This Li-ion battery, shown in Fig 3.8, powers the ESP8266 board and all peripherals. It provides a reliable source of energy and is rechargeable through standard charging circuits. With capacities between 1000–2000 mAh, it ensures several hours of operation without needing a recharge [40].

Specifications:

- Nominal Voltage: 3.7V
- Capacity: 1000–2000 mAh
- Charging Voltage: 4.2V max
- Discharge Current: 0.5C to 1C (typical)
- Dimensions: Varies (usually ~50 x 34 mm)

Fig 3.9 JST Connector 2-pin

Fig 3.10 12V 2A SMPS Adapter

JST Connector – 2-Pin Locking Type:

The JST connector, shown in Fig 3.9, provides a secure connection between the Li-ion battery and the circuit. Its locking mechanism prevents accidental disconnection, which is crucial for wearable or mobile applications. The compact design fits well on small PCBs [41].

JST Connector:

- Type: JST-XH or JST-PH
- Pin Count: 2

- Current Rating: ~2A
- Pitch: 2.0 mm or 2.5 mm
- Locking mechanism: Yes

12V 2A SMPS Adapter

The SMPS adapter, shown in Fig 3.10, provides a stable 12V DC output from a standard AC wall outlet. It is used externally to power development boards or to recharge the battery via a charge controller. Its lightweight, efficient design makes it ideal for lab and bench testing [42].

Specifications:

- Input Voltage: 100–240V AC
- Output Voltage: 12V DC
- Max Current Output: 2A
- Power Rating: 24W
- Output Plug: 5.5mm x 2.1mm barrel jack

Table 3.1 Summary of Hardware Components used

Sr. No.	Name of Component	Type	Function in System	Node Used
1	ESP8266 D1 Mini	Microcontroller	Core logic and communication	All Nodes
2	MAX30102	Biomedical Sensor	Monitors heart rate and oxygen saturation	Sentry Watch Node
3	MPU6050	Motion Sensor	Detects motion, fall, and posture	Sentry Watch Node
4	HC-SR501 PIR Sensor	Motion Detection Sensor	Detects intrusions and motion activity	Motion Detection Nodes
5	OLED SSD1306	Display Module	Displays system status and alerts	Control Dashboard Node
6	Mini 360 Buck Converter	Power Regulator	Steps down voltage for safe component operation	All Nodes
7	Active Buzzers	Alert Output Device	Emits sound alerts on event detection	Watch & Dashboard Nodes
8	Li-ion Battery 3.7V	Power Supply	Portable power source	Watch & Motion Nodes
9	JST 2-Pin Connector	Power Connector	Connects battery safely to PCB	Watch & Motion Nodes
10	12V 2A SMPS Adapter	External Adapter	Provides input power or charges battery	Dashboard Node

3.4 Sentry Watch Node

The Sentry Watch Node, shown in Fig 3.11, is built around the ESP8266 (NodeMCU) microcontroller. It integrates the following components:

- MAX30102 sensor for real-time heart rate and SpO_2 monitoring.
- MPU6050 6-axis accelerometer and gyroscope for detecting posture, orientation, and motion.
- SSD1306 OLED Display to provide immediate visual feedback on system status and connection.
- Dual piezo buzzers to raise alerts when a fainting or abnormal biometric state is detected.

Sensor data is processed locally on the ESP8266 and transmitted via HTTP POST to a central dashboard hosted on the Control Node. The entire unit is powered using a Li-ion battery, regulated through a Mini 360 buck converter and Low Dropout Regulator (LDO) for safe and consistent voltage delivery.

3.5 Motion Detection Nodes

The system incorporates three independent Motion Detection Nodes, each based on the ESP8266 (NodeMCU) and connected to an HC-SR501 PIR sensor, as illustrated in Fig 3.12. These nodes are strategically deployed in separate zones. To prevent communication interference and reduce false triggers:

- Only one node remains active at a time.
- Detected motion is sent to the Control Node via ESP-NOW, a lightweight, peer-to-peer protocol.

This architecture allows for low-latency detection without reliance on internet infrastructure, making it suitable for remote security applications.

3.6 Control Dashboard Node (Sentry Panel)

The Control Dashboard Node, shown in Fig 3.13, is also based on an ESP8266 and serves as the central hub of the system. It performs multiple roles:

- Hosts a web dashboard using WebSocket and HTTP protocols.
- Displays live biometric states: Awake, Fainted, and Moving.
- Shows motion alerts with timestamped zone identification.
- Sends appropriate triggers to buzzers and the OLED screen for real-time alerting.

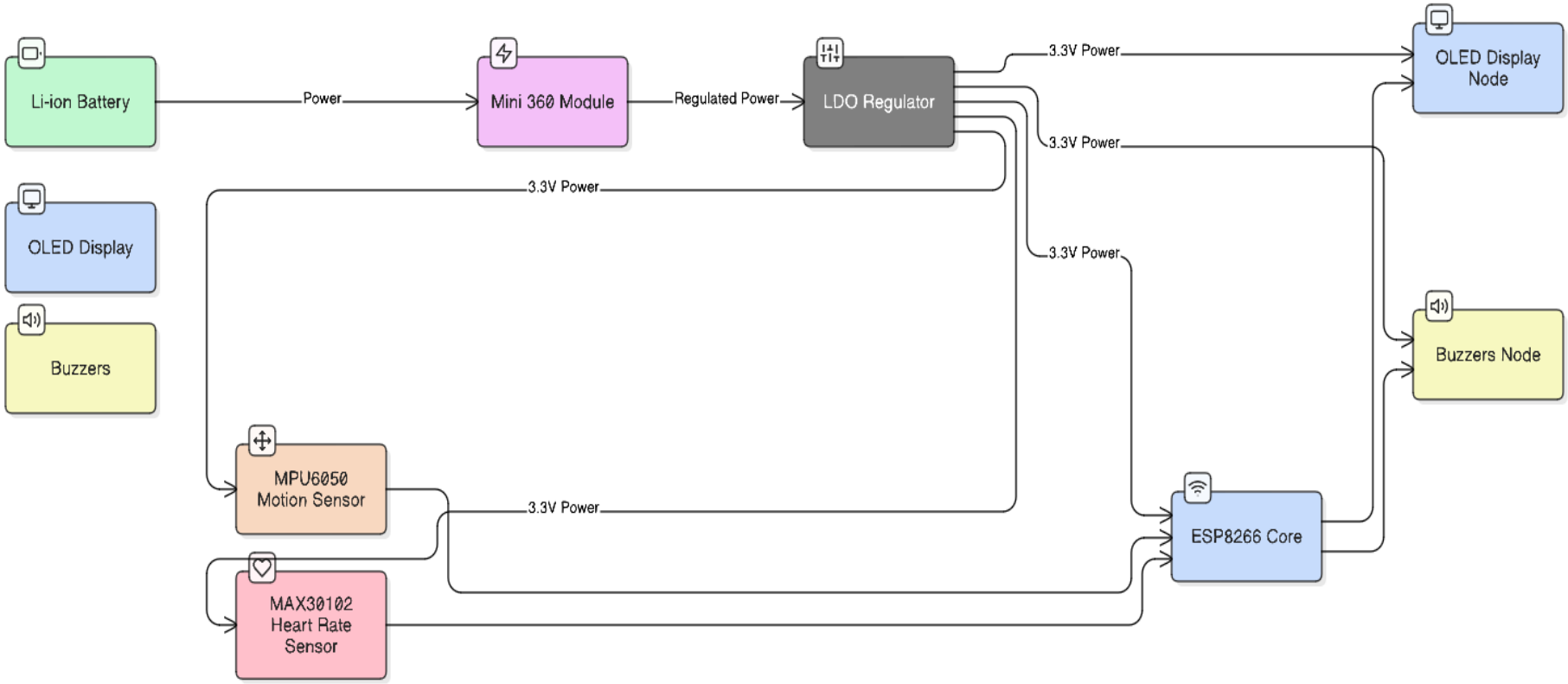

Fig 3.11 – Flow Diagram of Health (Watch) Node

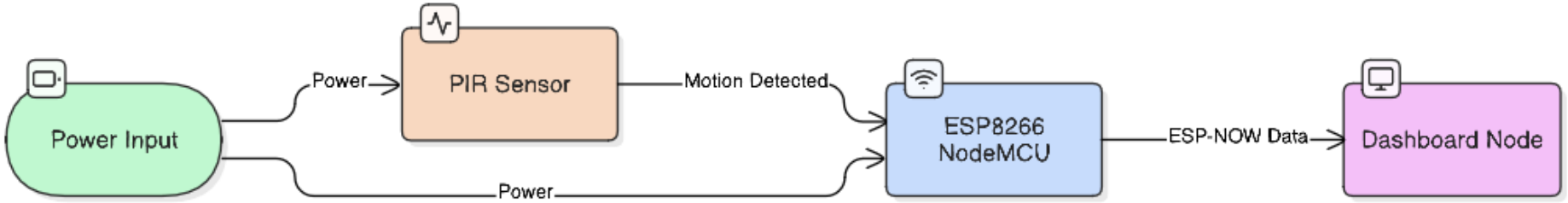

Fig 3.12 – Flow Diagram of Motion Node

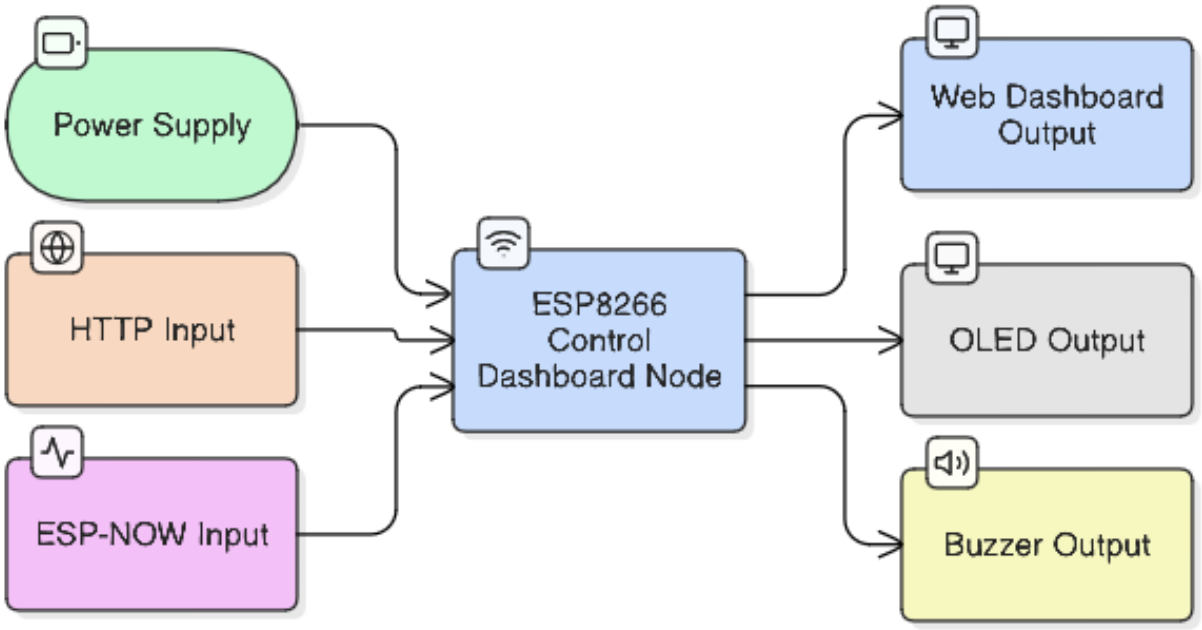

Fig 3.13 Flow Diagram of Control Node

3.7 Communication Flow

The communication flow between nodes is structured to ensure low-latency, local processing without dependence on external servers:

- Watch Node → Dashboard: Via HTTP POST (biometric data: BPM, motion status).
- Motion Nodes → Dashboard: Via ESP-NOW (motion alerts).
- Dashboard → OLED + Buzzers: Through internal logic using GPIO-based triggering.

This hybrid approach balances robustness with simplicity in real-time operations.

3.8 Power and Packaging

All nodes are built using compact, low-power PCBs with JST locking connectors to ensure safety, modularity, and ease of maintenance. Specific design highlights include:

- The watch node is enclosed in a wearable strap-based prototype for mobility.
- Motion nodes are enclosed in durable, compact plastic housings to support zone-based deployment.

Efficient power management, portability, and rugged enclosures make the system suitable for outdoor or mission-critical field operations.

3.9 Circuit Schematic Description of Control Node

The Control Node, shown in Fig 3.14 in this system is built around a Wemos D1 Mini (ESP8266) microcontroller, as shown in Figure 3.4. It is responsible for issuing alerts when either abnormal vitals or motion inactivity is detected.

- Two buzzers (BZ1 and BZ2) are connected in parallel to GPIO pin D6 of the D1 Mini. This setup ensures that both buzzers can be triggered simultaneously for a stronger audible alert when the user faints or no motion is detected.
- An OLED display (OLED1) is connected via the I2C interface, with SCL connected to D1 and SDA to D2. The OLED displays live feedback including heart rate, motion status, and warning prompts such as “Respond” or “Fainted.”
- Power to the node is supplied through the 5V and GND pins of the Wemos board. Input power from a Li-ion battery is regulated through a Mini 360 buck converter followed by a Low Dropout (LDO) regulator to ensure safe and stable output for the OLED and buzzers.

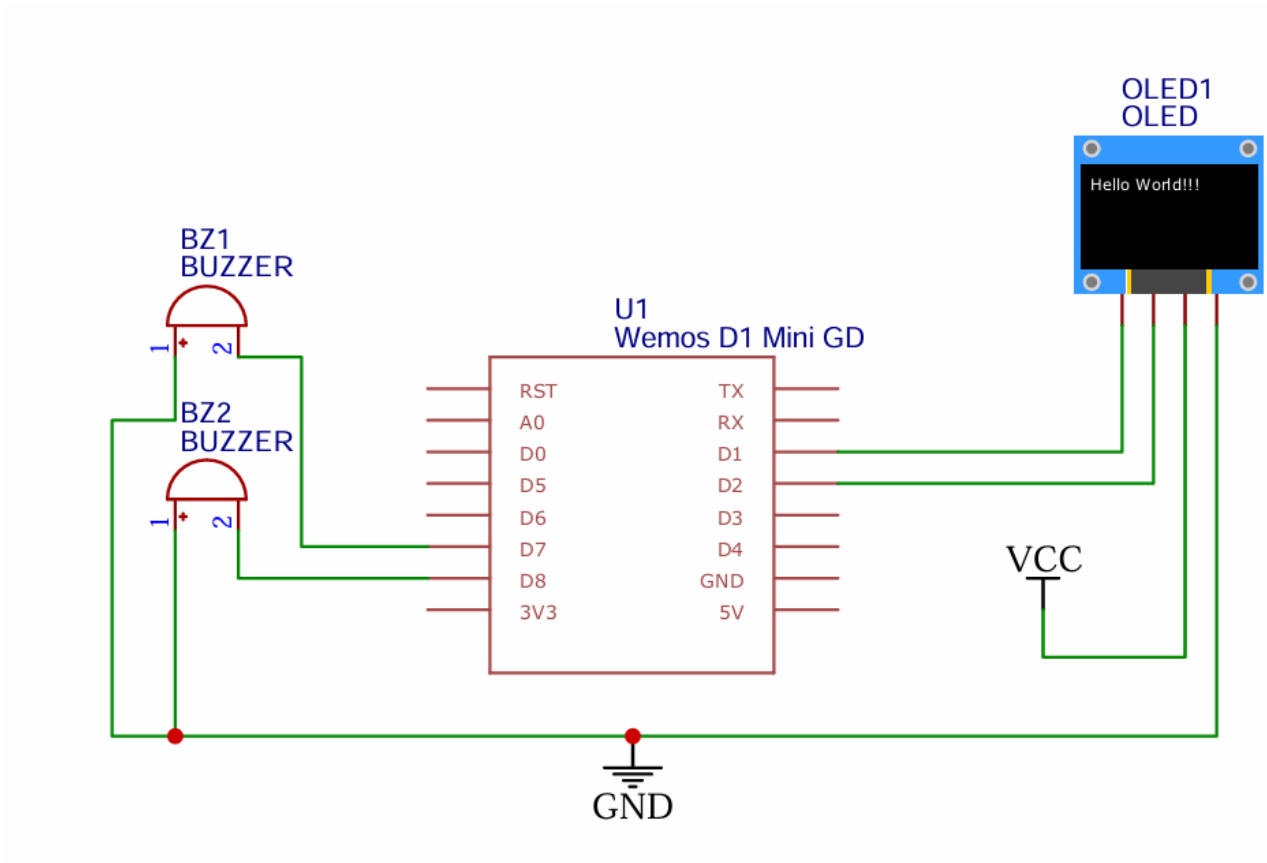

Fig 3.14 – Schematic of Control Node

3.10 Circuit Schematic Description of Motion Node

The Motion Node schematic, shown in Fig 3.15, also uses a Wemos D1 Mini.

- It integrates a PIR sensor (U2) with three pins:
  - VCC (pin 1) → connected to 5V output
  - GND (pin 2) → connected to ground
  - OUT (pin 4) → connected to GPIO pin D5 of the Wemos D1 Mini
- When motion is detected, the PIR sensor outputs a HIGH signal to D5, which is read by the ESP8266 and transmitted via ESP-NOW to the Control Node.
- Power for this node is also regulated through a Mini 360 DC-DC converter (U3). It steps down voltage from a Li-ion battery source, delivering stable 5V to both the PIR sensor and the ESP8266.
- The PIR output is polled continuously, and if inactivity exceeds a preset threshold, the node signals the Control Node to trigger visual and audible alerts.

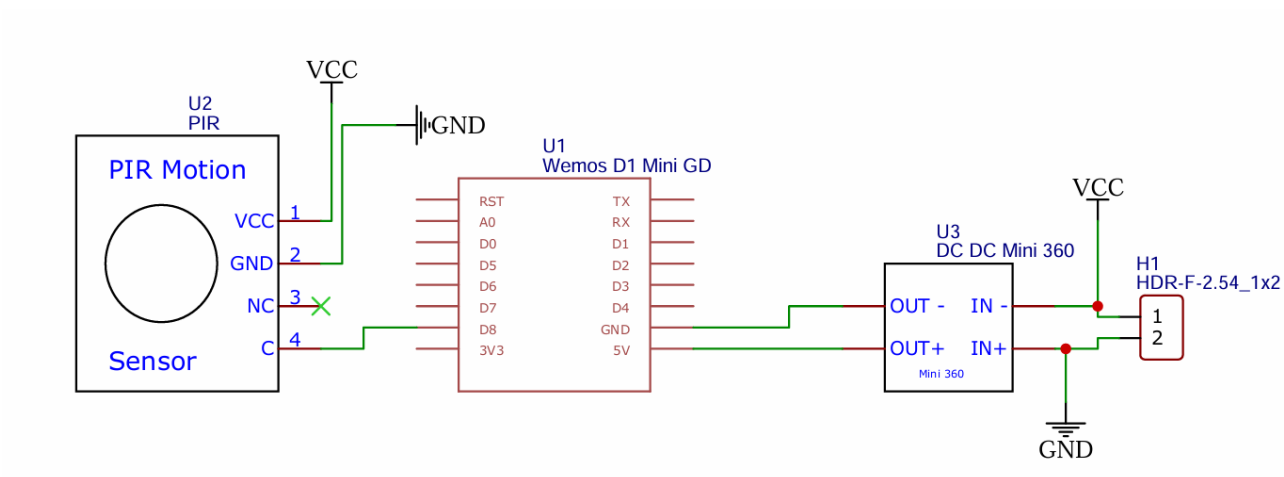

Fig 3.15 – Schematic of Motion Node

These schematic diagrams clearly represent how key components are interconnected and powered, and they support the modular structure necessary for deployment in field or wearable configurations.

Chapter 4
Prototype Deployment and Results

Chapter Outline

This chapter presents the physical deployment and validation of the IoT-based Sentry Alert and Monitoring System. It documents the setup of the wearable watch node, motion detection nodes, and the control dashboard. Real-time testing was conducted to evaluate motion detection, health status monitoring, and alert response. Output interfaces such as the OLED display, buzzer, and WebSocket-based dashboard are analyzed, and their responsiveness is discussed. Finally, a performance review and justification of achieved objectives are provided.

4.1 Implementation Overview

The prototype integrates three primary subsystems: wearable health monitoring (watch node), perimeter motion sensing (motion nodes), and a centralized control dashboard. ESP8266-based microcontrollers form the backbone of each node, handling sensor input and wireless data transmission. The watch node gathers real-time vitals using MAX30102 and MPU6050 sensors, while the motion nodes detect intrusions using PIR modules. These nodes transmit data via ESP-NOW to a central control unit, which triggers alerts, updates an OLED screen, and logs information on the dashboard.

The ESP modules were selected for their small footprint, integrated Wi-Fi support, and compatibility with real-time communication protocols, making them ideal for distributed wearable and fixed deployments.

4.2 Hardware Deployment Results

The final prototype was structured into three main functional units, each responsible for a specific aspect of monitoring and alerting:

- Motion Detection Nodes as shown in Fig 4.1, These are strategically placed in different perimeter zones to monitor intrusion. To minimize interference and reduce power consumption, only one node is active at a time. Each node uses a PIR (Passive Infrared) sensor to detect motion based on thermal changes in the environment. Upon activation, it immediately communicates with the control node via ESP-NOW, a low-latency communication protocol.

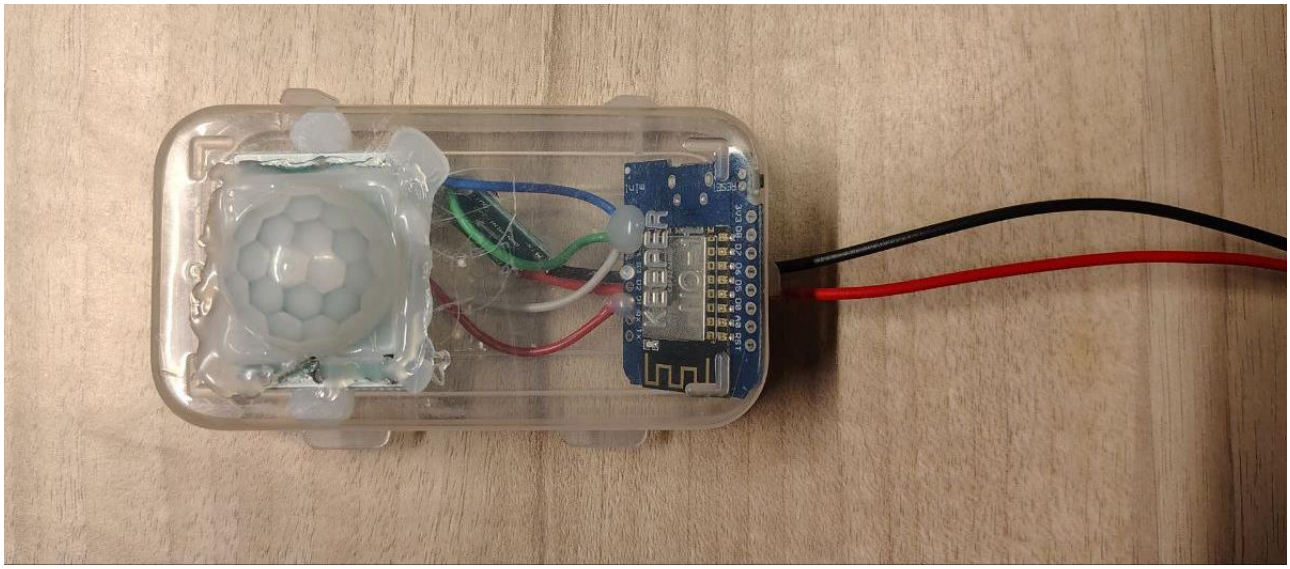

Fig 4.1 Prototype IOT System for Motion Node

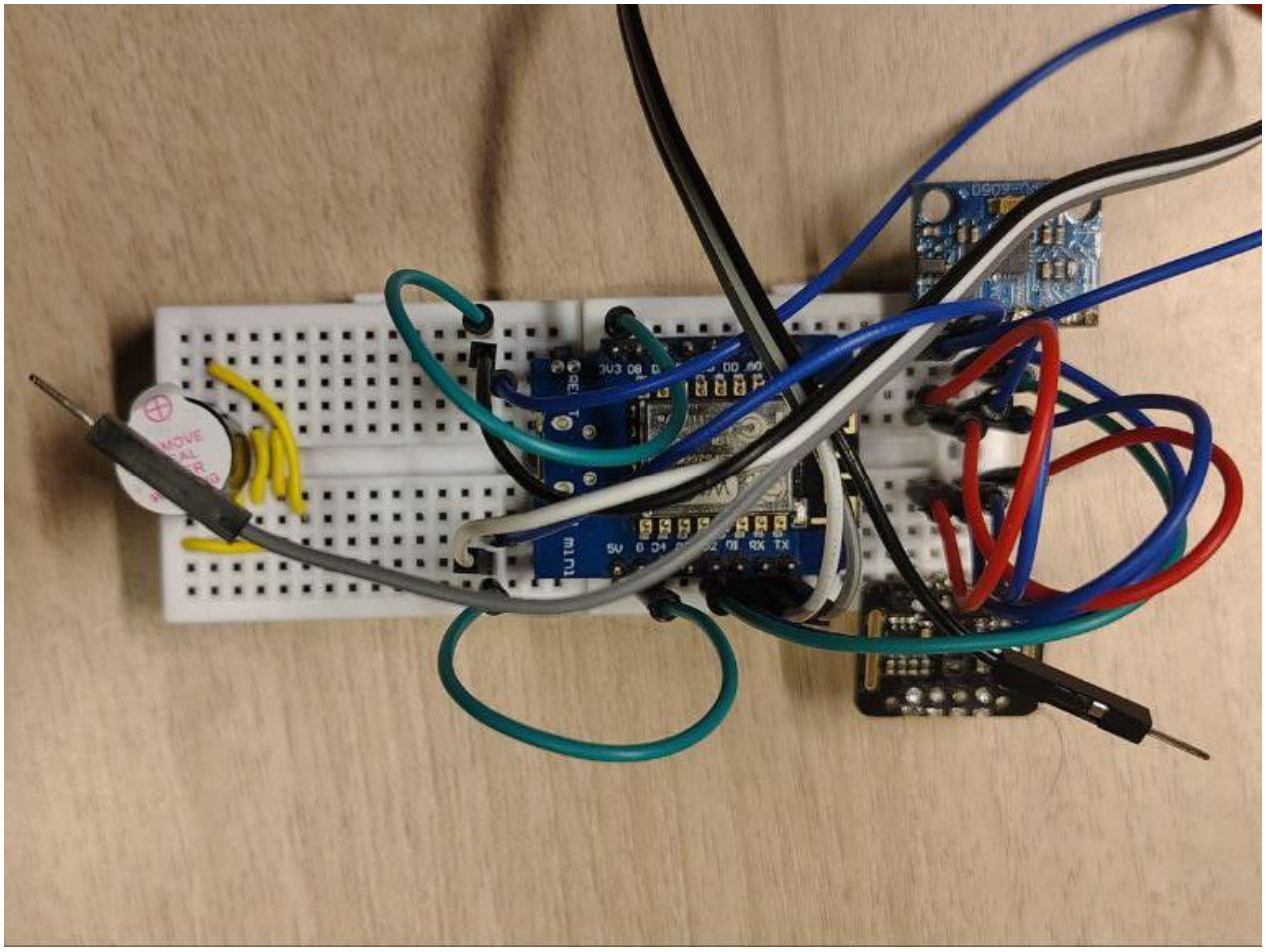

Fig 4.2 Prototype IOT System for Watch Node

- Watch Node as shown in Fig 4.2, This wearable unit is strapped onto the soldier or sentry and continuously monitors biometric parameters using a combination of the MAX30102 pulse oximeter and MPU6050 accelerometer-gyroscope module. The integration of these

sensors allows the device to detect states such as Awake, Moving, or Fainted, based on movement and vital sign thresholds.

- Control Dashboard Node as shown in Fig 4.3 Acting as the command hub, this node aggregates all incoming data via ESP-NOW and WebSocket protocols. It visualizes real-time statuses on a web-based dashboard and triggers buzzer alerts when an abnormal condition is detected. It also pushes messages to an OLED display to provide visual feedback to local personnel.

Fig 4.3 Prototype IOT System for Sentry Panel (Control Node)

4.3 Motion Detection Testing

The motion detection units were subjected to simulated intrusion events in varying environments to evaluate sensitivity, accuracy, and system latency. Each test followed a predefined sequence:

- A person walked past or gestured in front of the active PIR node.
- Upon detection, the ESP-12F module transmitted the alert using ESP-NOW to the control node.
- The WebSocket dashboard received and displayed a live motion alert as shown in Fig 4.4, indicating the zone number and status and No motion alert as shown in Fig 4.5.

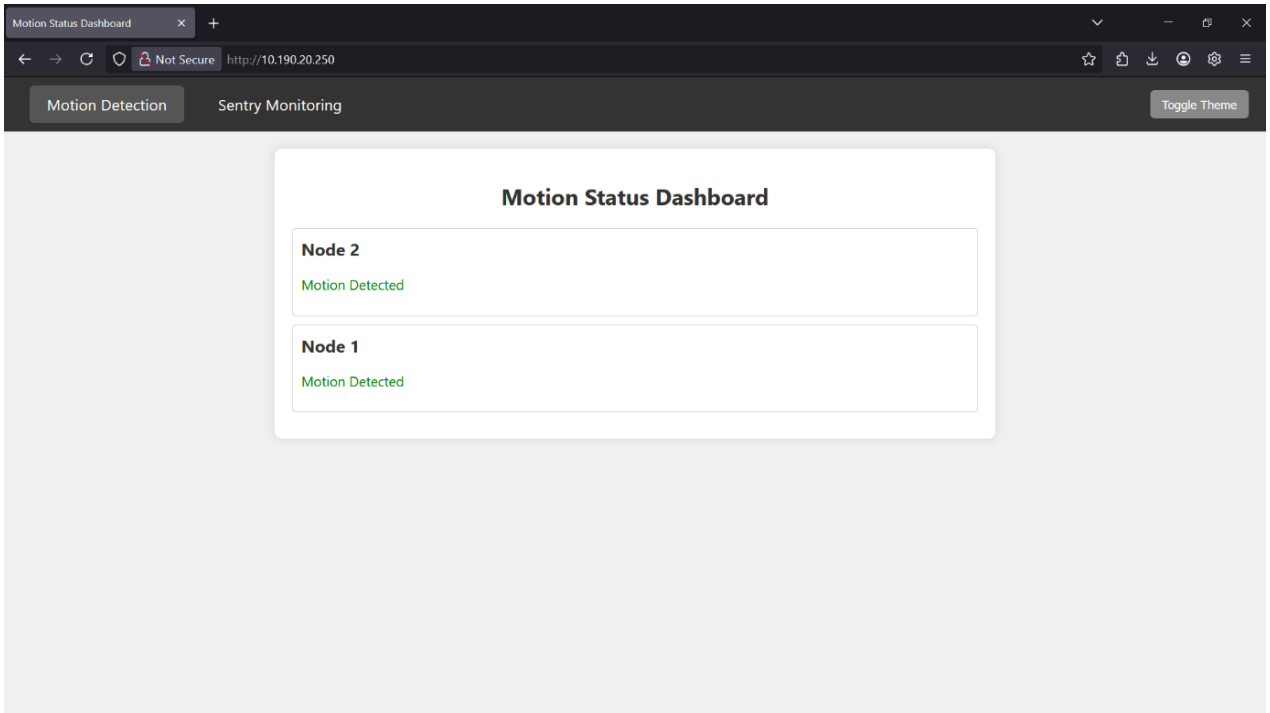

Fig 4.4 Server Dashboard indicating Motion Detected

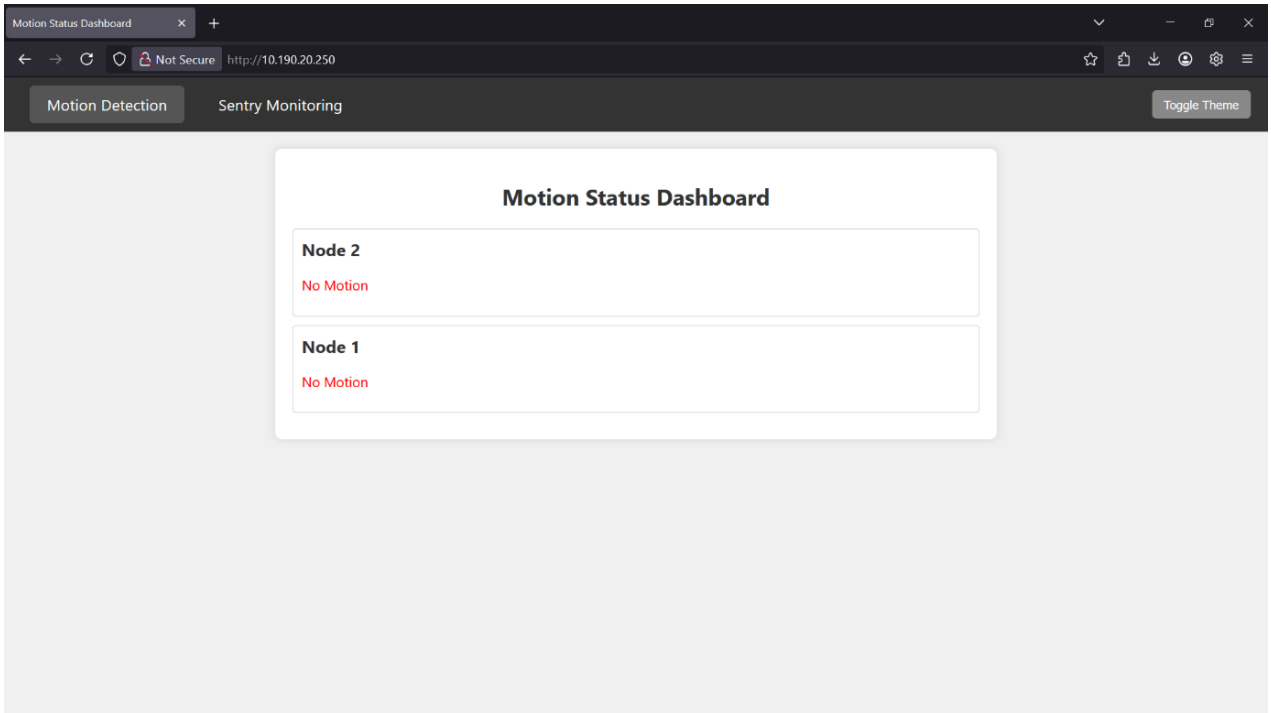

Fig 4.5 Server Dashboard indicating No Motion

- Simultaneously, the OLED display showed a corresponding message as shown in Fig 4.6, providing immediate on-site feedback.
- A buzzer was triggered to alert command staff through audio signals.

Tests were conducted under various lighting and ambient temperature conditions to assess the reliability of the PIR sensor. The system consistently demonstrated fast and accurate detection with no spurious activations in static environments.

Fig 4.6 – Motion Alert Display on OLED

4.4 Vital Monitoring Testing

The wearable watch node was tested for its ability to recognize and differentiate between critical physical states by interpreting biometric and motion sensor data :

- The MAX30102 sensor monitored heart rate and SpO_2 levels.
- The MPU6050 detected movement, posture, and orientation changes.

From this data, the system was programmed to recognize three distinct states:

- Awake: Normal heart rate with periodic motion signals as shown in Fig 4.7
- Fainted: Prolonged inactivity along with critical heart rate values as shown in Fig 4.8
- Moving: Detectable acceleration/motion with healthy vitals as shown in Fig 4.9

If the node identified a “Fainted” condition, it transmitted a critical alert to the control node, which:

- Updated the status on the WebSocket dashboard interface.
- Triggered a buzzer for urgent notification.
- Marked the user state visually, aiding quick decision-making.

The accuracy of this logic was verified through controlled scenarios where the user was asked to remain still, walk, or simulate fainting by lying motionless for extended durations.

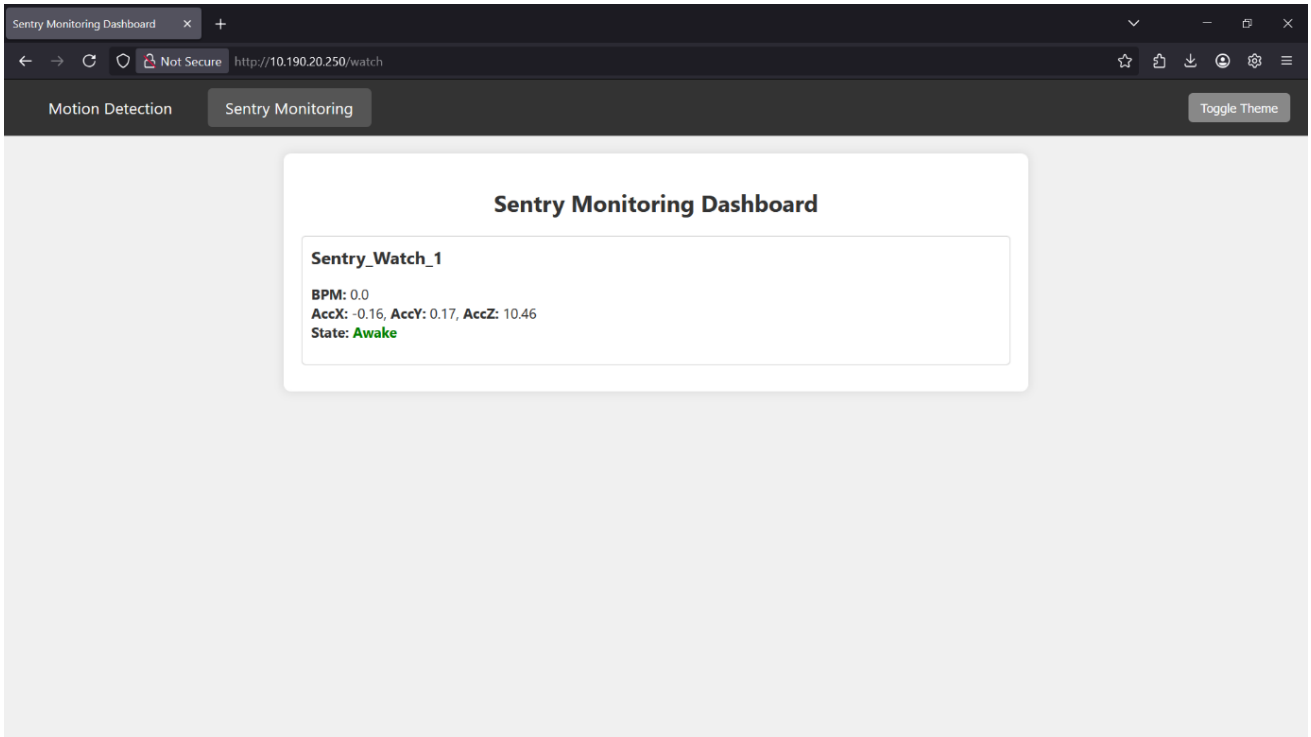

Fig 4.7 Server Dashboard indicating Awake Status

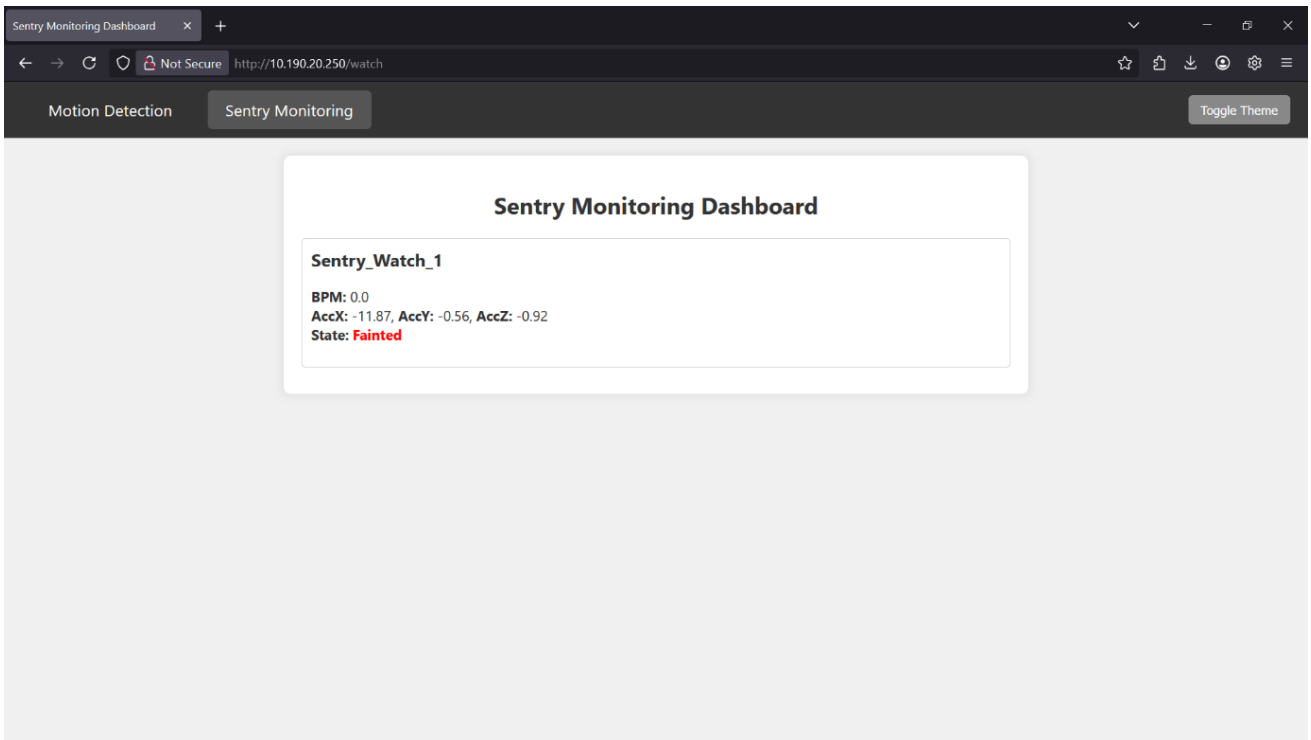

Fig 4.8 Server Dashboard indicating Fainted Status

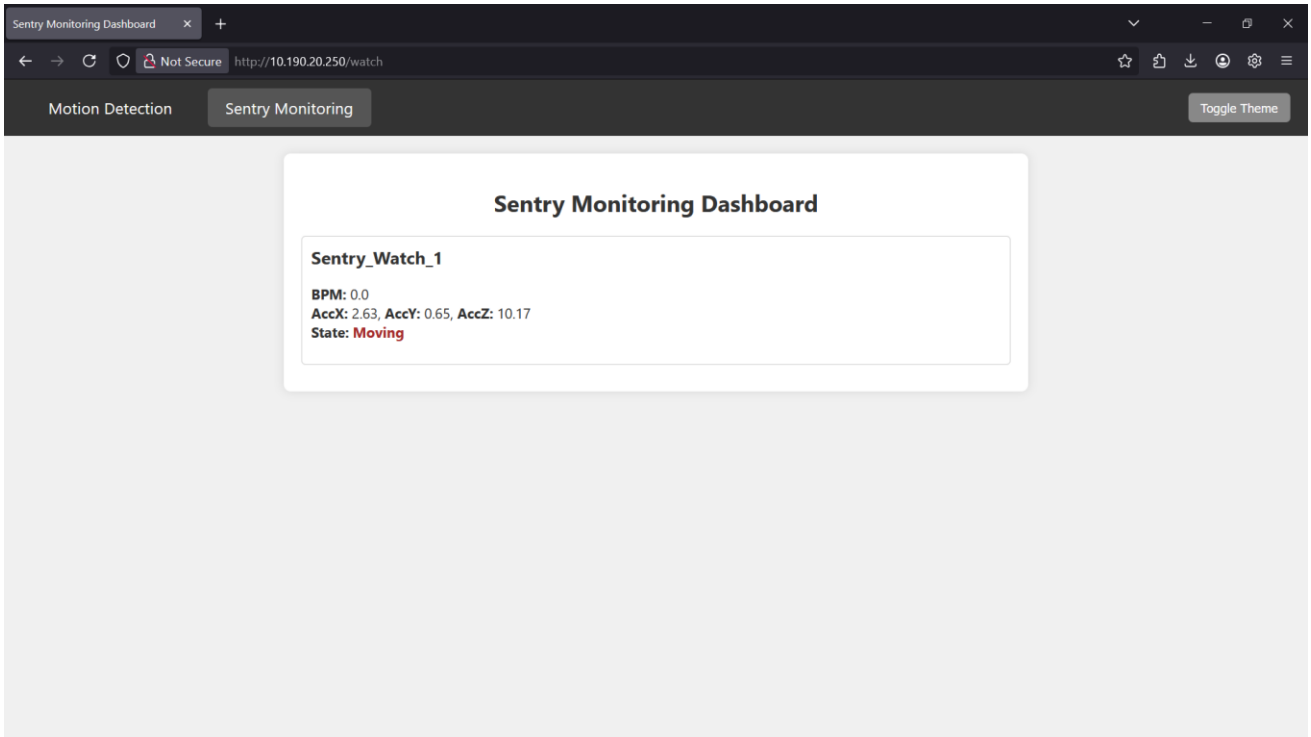

Fig 4.9 Server Dashboard indicating Moving Status

4.5 Result Analysis

Comprehensive testing was performed under controlled laboratory conditions to evaluate the system's performance across all three nodes. The results were assessed using the following criteria:

- Latency: The control dashboard updated motion and health status within 300–500 milliseconds of event occurrence, confirming low-latency communication via ESP-NOW and WebSocket protocols.
- Reliability: During repeated motion tests and biometric simulations, no false positives were recorded. All events were logged accurately and displayed on the OLED and dashboard interfaces.
- Synchronization: The motion alert, buzzer sound, and OLED messages were observed to activate nearly simultaneously, proving high synchronization among modules.
- Data Integrity: Continuous cycling of events for extended periods showed that all data packets were received without loss or corruption, validating the robustness of the wireless communication stack.
- Sensor Precision: Heart rate readings remained consistent within ±3 BPM compared to commercial pulse oximeters, and motion detection accuracy exceeded 95% in static background conditions.

4.6 Discussions and Inferences Drawn

The integrated system proved that combining wearable health monitoring with zonal motion detection substantially improves surveillance efficiency and personnel safety. Real-time alerts allowed for:

- Immediate response to potential medical emergencies.
- Rapid notification of unauthorized entry or motion.
- Enhanced decision-making through centralized visualization.

Key observations include:

- Modular hardware design enabled fast deployment and relocation.
- The use of ESP-NOW allowed communication in remote or offline environments.
- The dual alert mechanism (buzzer + OLED) minimized the chances of missed alerts.

In essence, the system demonstrated that proactive monitoring can significantly reduce the burden on human vigilance, improving overall situational awareness in field operations.

4.7 Justification of Objectives Achieved

The project set out with three primary objectives—all of which were successfully fulfilled through the developed prototype:

- Wearable Monitoring Achieved: The watch node, powered by the MAX30102 and MPU6050, was able to provide real-time monitoring of physiological parameters and detect fatigue or collapse conditions.
- Motion Detection Functionality: PIR-based motion nodes effectively detected human movement across designated zones and transmitted data promptly with zero observed interference.
- Real-Time Centralized Dashboard: The integration of a live, browser-based dashboard using WebSockets allowed command staff to receive instant updates on both biometric and motion events, improving operational readiness.

These results confirm that the system is not only technically sound but also practically applicable for real-world deployment in defense or security-sensitive zones.

Chapter 5
Conclusion and Future Work

5.1 Conclusions

This project successfully developed a real-time IoT-based Sentry Alert and Monitoring System that integrates wearable health tracking and perimeter motion detection into a cohesive, edge-operated solution. The system employs ESP8266-based microcontrollers in conjunction with the MAX30102 sensor for heart rate and SpO_2 monitoring, MPU6050 for movement and orientation sensing, and HC-SR501 PIR sensors for motion detection across different zones.

Communication between nodes was achieved using a hybrid model:

- ESP-NOW protocol enabled low-latency, peer-to-peer messaging for motion alerts.
- HTTP POST and WebSocket protocols facilitated reliable, two-way data transfer between the wearable node and the control dashboard.

The system effectively recognized and responded to three key user states—Awake, Fainted, and Moving—by generating alerts through buzzers, an OLED display, and a web-based dashboard interface. The design remains cloud-independent, enabling offline operation with minimal latency, which is essential for deployment in remote or high-security environments.

Its modular architecture allows for easy expansion or reconfiguration, making it suitable for a variety of use cases beyond military applications, such as elderly health monitoring, industrial safety, or personal surveillance systems. By integrating health and security monitoring into a single system, the project presents a cost-effective, scalable, and practical solution to address traditional shortcomings in manual and disconnected surveillance systems.

.

5.2 Future Work

Many different enhancements, integrations, and field-level tests have been set aside for future development due to limitations in time and resources. The real-time performance validation and stress testing of such a system, especially in dynamic environments like military zones, often require extended durations and carefully controlled conditions. Future work will focus on refining system performance, improving component-level efficiency, and expanding the system's adaptability to meet evolving defense requirements.

The following points outline specific areas for further development:

- Integrate fatigue analysis algorithms using machine learning models to improve the accuracy of physical stress detection and early warning mechanisms.
- Upgrade the existing server node to a more robust platform capable of handling multi-node connections and processing large datasets for real-time analytics.
- Introduce encrypted communication protocols to enhance the security of transmitted health and motion data within the network.
- Improve the motion detection system's sensitivity and precision by integrating advanced filtering techniques to reduce false alerts in high-noise environments.
- Design a modular hardware framework for easier maintenance, upgrades, and deployment across varied terrain and climatic conditions.
- Develop a lightweight, rugged wearable enclosure to improve comfort, durability, and long-term usability for soldiers on active duty.

REFERENCES

[1] A. Al-Fuqaha et al., "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," IEEE Commun. Surv. Tutor., vol. 17, no. 4, pp. 2347–2376, 2015.

[2] M. Ammar, G. Russello, and B. Crispo, "Internet of Things: A Survey on the Security of IoT Frameworks," J. Inf. Secur. Appl., vol. 38, pp. 8–27, 2018.

[3] K. Ashton, "That 'Internet of Things' Thing," RFID Journal, 2009. [Online]. Available: https://www.rfidjournal.com/that-internet-of-things-thing

[4] B. Silva, M. Khan, and K. Han, "Wearable Sensors for Monitoring Physical Activity and Health," IEEE Sensors J., vol. 15, no. 3, pp. 1321–1330, 2015.

[5] C. Doukas and I. Maglogiannis, "Bringing IoT and Cloud Computing towards Pervasive Healthcare," in Proc. IMIS, 2012, pp. 922–926.

[6] M. Zao, A. Wang, S. Pande, and M. B. Srivastava, "BodyBeat: A Mobile System for Sensing Non-Speech Body Sounds," in Proc. SenSys, 2014, pp. 151–165.

[7] Espressif Systems, "ESP8266 Technical Reference Manual," 2020. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp8266-technical_reference_en.pdf

[8] R. Want and T. Pering, "System Challenges for Ubiquitous & Invisible Computing," Proc. IEEE, vol. 100, no. Centennial Issue, pp. 622–633, May 2012.

[9] Espressif Systems, "ESP-NOW User Guide," 2021. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html

[10] J. Banks and D. T. Jones, "WebSocket Technology and Real-Time IoT Dashboards," IEEE Internet Comput., vol. 22, no. 4, pp. 62–68, 2018.

[11] M. Collotta and G. Pau, "A Novel Approach for Dynamic Power Management in IoT-Based Smart Home," IEEE Trans. Syst. Man Cybern., vol. 47, no. 1, pp. 93–105, 2017.

[12] A. Thangavel et al., "Performance Analysis of WebSocket Protocol for Real-Time Web Applications," Procedia Comput. Sci., vol. 50, pp. 120–125, 2015.

[13] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog Computing and Its Role in the Internet of Things," in Proc. MCC Workshop Mobile Cloud Comput., 2012, pp. 13–16.

[14] R. Bajaj et al., "Real-Time Health Monitoring Using Mobile Devices and Wearables," IEEE Consumer Electron. Mag., vol. 6, no. 4, pp. 38–44, 2017.

[15] S. Misra, P. V. Krishna, H. Agarwal, A. Saxena, and M. S. Obaidat, "A Learning Automata Based Solution for Preventing Distributed Denial of Service in Internet of Things," Int. J. Commun. Syst., vol. 31, no. 5, pp. e3491, 2018.

[16] D. Evans, "The Internet of Things: How the Next Evolution of the Internet Is Changing Everything," Cisco, White Paper, 2011.

[17] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of Things for Smart Cities," IEEE Internet Things J., vol. 1, no. 1, pp. 22–32, Feb. 2014.

[18] S. M. Riazul Islam, D. Kwak, M. Humaun Kabir, M. Hossain, and K.-S. Kwak, "The Internet of Things for Health Care: A Comprehensive Survey," IEEE Access, vol. 3, pp. 678–708, 2015.

[19] A. Pantelopoulos and N. G. Bourbakis, "A survey on wearable sensor-based systems for health monitoring and prognosis," IEEE Trans. Syst., Man, Cybern. C, Appl. Rev., vol. 40, no. 1, pp. 1–12, Jan. 2010.

[20] S. Majumder, T. Mondal, and M. J. Deen, "Smart homes for elderly healthcare—Recent advances and research challenges," Sensors, vol. 17, no. 11, pp. 1–26, 2017.

[21] D. Y. Lee and Y. S. Suh, "Improvement of positioning accuracy using a wearable inertial sensor system for soldiers," Sensors, vol. 10, no. 10, pp. 9363–9378, 2010.

[22] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A vision, architectural elements, and future directions," Future Gener. Comput. Syst., vol. 29, no. 7, pp. 1645–1660, 2013.

[23] Y. Zhao and S. C. Cheung, "Human activity recognition using spatial-temporal features and motion energy image," in Proc. IEEE VS-PETS, Rio de Janeiro, Brazil, 2007, pp. 1–8.

[24] S. Singh, S. K. Tanwar, and R. Kumar, "An intrusion detection system using various machine learning algorithms: A review," Int. J. Security Appl., vol. 8, no. 4, pp. 247–256, 2017.

[25] Z. Ali, F. Mehmood, M. I. Khan, and M. H. Alizai, "IoT-based smart health monitoring system for COVID-19 patients," Int. J. Pervasive Comput. Commun., vol. 14, no. 4, pp. 1–12, 2018.

[26] V. Paxson, "Bro: A system for detecting network intruders in real-time," Comput. Netw., vol. 31, no. 23–24, pp. 2435–2463, 1999.

[27] J. Yick, B. Mukherjee, and D. Ghosal, "Wireless sensor network survey," Comput. Netw., vol. 52, no. 12, pp. 2292–2330, 2008.

[28] M. Chen, Y. Ma, J. Song, C. Lai, and B. Hu, "Wearable 2.0: Enabling human-cloud integration in next generation healthcare systems," IEEE Commun. Mag., vol. 55, no. 1, pp. 54–61, Jan. 2017.

[29] A. Kumar and M. Patel, "A review on real-time dashboard for healthcare using IoT," Int. J. Comput. Sci. Inf. Technol., vol. 5, no. 3, pp. 4985–4987, 2014.

[30] J. N. Al-Karaki and A. E. Kamal, "Routing techniques in wireless sensor networks: A survey," IEEE Wireless Commun., vol. 11, no. 6, pp. 6–28, Dec. 2004.

[31] M. S. and G. Muhammad, "Cloud-assisted industrial internet of things (IIoT)–Enabled framework for health monitoring," IEEE Internet Things J., vol. 3, no. 5, pp. 1–10, 2016.

[32] N. Bhatia, V. K. Singh, and S. Sharma, "Recent advances in smart wearable technologies for monitoring soldier health and surveillance," Def. Sci. J., vol. 71, no. 1, pp. 1–10, 2021.

[33] Espressif Systems, ESP8266EX Datasheet, Rev. 1.1, Espressif Systems, Aug. 2020. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/0a-esp8266ex_datasheet_en.pdf Accessed: Feb. 22, 2025, 9:12 AM.

[34] InvenSense, MPU-6000 and MPU-6050 Product Specification, Rev. 3.4, 2013. [Online]. Available:https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000 Datasheet1.pdf Accessed: Mar. 9, 2025, 4:27 PM.

[35] Maxim Integrated, MAX30102 Pulse Oximeter and Heart-Rate Sensor for Wearable Health, Rev. 2, Jan. 2016. [Online]. Available: https://datasheets.maximintegrated. com/en/ds/MAX30102.pdf Accessed: Apr. 1, 2025, 6:53 PM.

[36] Hi-Link, Mini 360 Buck Converter Datasheet, HLK-PM Series. [Online]. Available: https://components101.com/sites/default/files/component_datasheet/HLK-PM01%20Datasheet.pdf Accessed: Mar. 25, 2025, 8:37 AM.

[37] Adafruit Industries, HC-SR501 PIR Motion Sensor Module, Product Datasheet. [Online]. Available:https://cdn-learn.adafruit.com/assets/assets/000/017/748/original/PIRsensor.pdf Accessed: May 8, 2025, 2:41 PM.

[38] Waveshare, 0.96 inch OLED Display Module Datasheet, SSD1306 Driver. [Online]. Available: https://www.waveshare.com/wiki/0.96inch_OLED_Module Accessed: Apr. 16, 2025, 7:09 AM.

[39] Murata, Piezoelectric Sound Components: PKM13EPYH4002-B0, Aug. 2021. [Online]. Available:https://www.murata.com/enus/products/productdata/8807031627806/MURATA_PIEZO_BUZZER_DATASHEET.pdf Accessed: Mar. 2, 2025, 11:24 AM.

[40] Panasonic, Li-ion Rechargeable Batteries NCR18650B, Tech Data Sheet, 2014. [Online]. Available:https://www.panasonic.com/global/energy/product/pdf/ncr18650b.pdf Accessed: May 3, 2025, 1:16 PM.

[41] JST Mfg. Co., Ltd., PH Series 2-Pin Connectors Datasheet. [Online]. Available: https://www.jst-mfg.com/product/pdf/eng/ePH.pdf Accessed: Feb. 29, 2025, 5:58 PM.

[42] Mean Well Enterprises, GS12U12-P1J – 12V 2A SMPS Adapter Datasheet, Rev. B, 2020. [Online]. Available: https://www.meanwell-web.com/en-gb/ac-dc-plug-in-adapter-output-12vdc-at-2a-gs12u12-p1j Accessed: Apr. 10, 2025, 12:03 PM.

ANNEXURE A

A.1 CPP PROGRAM CODE FOR MOTION NODE

```
#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>

// === CONFIGURATION ===
#define NODE_ID "Node 1"  // Change this to "Node 1", "Node 3", etc.
const char* ssid = "Anakin";
const char* password = "Skywalker31";
const char* serverIP = "10.190.20.250";  // Dashboard ESP IP
const int motionPin = D1;

unsigned long clearDelay = 2000;         // Time before declaring "No Motion"
unsigned long resendInterval = 5000;     // Optional: Resend motion if it's persisting
// ======================

bool lastMotionState = LOW;
unsigned long lastMotionTime = 0;
unsigned long lastSentTime = 0;

void setup() {
  Serial.begin(115200);
  pinMode(motionPin, INPUT);

  WiFi.mode(WIFI_STA);
  WiFi.setSleepMode(WIFI_NONE_SLEEP);  //  Prevent sleep delay
  WiFi.begin(ssid, password);

  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
```

```
  }
  Serial.println("\n Connected. IP: " + WiFi.localIP().toString());
}

void loop() {
  bool motionNow = digitalRead(motionPin);
  unsigned long now = millis();

  // New motion detected (rising edge)
  if (motionNow == HIGH && lastMotionState == LOW) {
    lastMotionState = HIGH;
    lastMotionTime = now;
    lastSentTime = now;
    sendMotionStatus("motion");
  }

  // Still motion, resend if motion continues for too long
  if (motionNow == HIGH && lastMotionState == HIGH && now - lastSentTime >
resendInterval) {
    lastSentTime = now;
    sendMotionStatus("motion");  // Persistent motion
  }

  // No motion for a while
  if (lastMotionState == HIGH && now - lastMotionTime > clearDelay && motionNow ==
LOW) {
    lastMotionState = LOW;
    sendMotionStatus("pressed");
  }

  delay(20); // Faster response loop
}

void sendMotionStatus(const String& status) {
```

```
  if (WiFi.status() == WL_CONNECTED) {
    WiFiClient client;
    client.setTimeout(1000);

    HTTPClient http;
    String url = "http://" + String(serverIP) + "/motion?node=" + urlencode(NODE_ID) +
"&status=" + status;
    Serial.println("Sending: " + url);

    http.begin(client, url);
    int httpCode = http.GET();
    Serial.printf("HTTP Response Code: %d\n", httpCode);
    http.end();
  } else {
    Serial.println("WiFi not connected");
    reconnectWiFi();
  }
}

void reconnectWiFi() {
  WiFi.disconnect();
  delay(100);
  WiFi.begin(ssid, password);
  unsigned long start = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - start < 5000) {
    delay(500);
    Serial.print(".");
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\nReconnected to WiFi");
  } else {
    Serial.println("\nReconnect Failed");
  }
}
```

```
String urlencode(const String& str) {
  String encoded = "";
  for (unsigned int i = 0; i < str.length(); i++) {
    char c = str.charAt(i);
    if (c == ' ') {
      encoded += "%20";
    } else {
      encoded += c;
    }
  }
  return encoded;
}
```

A.2 CPP PROGRAM CODE FOR SENTRY PANEL

```
#include <ESP8266WiFi.h>
#include <WebSocketsClient.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,
OLED_RESET);

WebSocketsClient webSocket;

const char* ssid = "Anakin";
const char* password = "Skywalker31";

#define BUZZER_PIN1 D5
#define BUZZER_PIN2 D6
```

```
unsigned long motionDisplayTime = 0;
bool motionShown = false;
bool wifiConnected = false;

// Helper to center any text
void centerText(String text, int textSize, int y) {
  display.setTextSize(textSize);
  display.setTextColor(SSD1306_WHITE);
  int16_t x1, y1;
  uint16_t w, h;
  display.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
  int x = (SCREEN_WIDTH - w) / 2;
  display.setCursor(x, y);
  display.println(text);
}

void showMotion(String rawNodeId) {
  display.clearDisplay();

  // Extract number if already has "NODE" in it
  String cleanNode = rawNodeId;
  cleanNode.trim();
  cleanNode.toUpperCase();

  if (cleanNode.startsWith("NODE ")) {
    cleanNode = cleanNode.substring(5);  // Only get the number/ID part
  }

  String nodeText = "NODE " + cleanNode;

  centerText(nodeText, 2, 0);        // Big "NODE X"
  centerText("Motion", 1, 30);       // Smaller
  centerText("Detected", 1, 45);     // Smaller
```

```
  display.display();

  motionDisplayTime = millis();
  motionShown = true;
}

void showWiFiMessage(String msg, int duration = 0) {
  display.clearDisplay();

  if (msg == "WiFi Connected" || msg == "WiFi Disconnected") {
    String word1 = "WiFi";
    String word2 = (msg == "WiFi Connected") ? "Connected" : "Disconnected";

    centerText(word1, 2, 10);
    centerText(word2, 1, 35);
  } else {
    centerText(msg, 1, 25);
  }

  display.display();

  if (duration > 0) {
    delay(duration);
    display.clearDisplay();
    display.display();
  }
}

void webSocketEvent(WStype_t type, uint8_t * payload, size_t length) {
  if (type == WStype_TEXT) {
    String message = String((char*)payload);
    int sepIndex = message.indexOf(':');
    String node = message.substring(0, sepIndex);
    String status = message.substring(sepIndex + 1);
```

```
    if (status == "motion") {
      tone(BUZZER_PIN1, 1000);
      tone(BUZZER_PIN2, 1000);
      showMotion(node);
    } else if (status == "pressed") {
      noTone(BUZZER_PIN1);
      noTone(BUZZER_PIN2);
      // Clear after timeout
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(BUZZER_PIN1, OUTPUT);
  pinMode(BUZZER_PIN2, OUTPUT);

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED failed");
    while (true);
  }

  display.clearDisplay();
  display.display();

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  wifiConnected = true;
  Serial.println("WiFi Connected");
```

```
  showWiFiMessage("WiFi Connected", 2000);

  webSocket.begin("10.190.20.250", 81, "/"); // use your dashboard IP
  webSocket.onEvent(webSocketEvent);
  webSocket.setReconnectInterval(5000);
}

void loop() {
  webSocket.loop();

  if (motionShown && millis() - motionDisplayTime > 2500) {
    display.clearDisplay();
    display.display();
    motionShown = false;
  }

  if (WiFi.status() != WL_CONNECTED && wifiConnected) {
    showWiFiMessage("WiFi Disconnected");
    wifiConnected = false;
  } else if (WiFi.status() == WL_CONNECTED && !wifiConnected) {
    showWiFiMessage("WiFi Connected", 2000);
    wifiConnected = true;
  }
}
```

A.3 CPP PROGRAM CODE FOR WATCH NODE

```
#include <Wire.h>
#include <ESP8266WiFi.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <ESP8266HTTPClient.h>

const char* ssid = "Anakin";
const char* password = "Skywalker31";
```

```
const char* dashboardIP = "10.190.20.250"; // Update if different

Adafruit_MPU6050 mpu;
WiFiClient client;

#define BUZZER_PIN D5
#define NODE_ID "Sentry_Watch_1"

String currentState = "";

void setup() {
  Serial.begin(115200);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.println("Connecting to WiFi...");
  }
  Serial.println("Connected to WiFi");

  if (!mpu.begin()) {
    Serial.println("MPU6050 not found");
    while (1);
  }
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
  Serial.println("MPU6050 initialized");
}

void loop() {
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);
```

```
float accX = a.acceleration.x;
float accY = a.acceleration.y;
float accZ = a.acceleration.z;

float movement = abs(accX) + abs(accY) + abs(accZ - 9.8);

String state;

if (accZ < 0) {
  state = "Fainted"; // upside down
} else if (movement > 2.0) {
  state = "Moving";
} else {
  state = "Awake";
}

// Control buzzer
if (state == "Fainted") {
  digitalWrite(BUZZER_PIN, HIGH);
} else {
  digitalWrite(BUZZER_PIN, LOW);
}

// Send update if state changes or every 3s
static unsigned long lastSent = 0;
if (state != currentState || millis() - lastSent > 3000) {
  currentState = state;
  lastSent = millis();

  String url = "http://" + String(dashboardIP) + "/update?";
  url += "sensor=" + String(NODE_ID);
  url += "&accX=" + String(accX, 2);
  url += "&accY=" + String(accY, 2);
```

```
    url += "&accZ=" + String(accZ, 2);
    url += "&bpm=0";
    url += "&state=" + state;

    Serial.println("Sending: " + state);

    HTTPClient http;
    http.begin(client, url);
    int httpCode = http.GET();
    if (httpCode > 0) {
      Serial.println(http.getString());
    } else {
      Serial.println("HTTP failed");
    }
    http.end();
  }

  delay(500); // Fast update
}
```

A.4 CPP PROGRAM CODE FOR SERVER NODE

```
#include <ESP8266WiFi.h>
#include <ESPAsyncWebServer.h>
#include <WebSocketsServer.h>
#include <ESP8266mDNS.h>

const char* ssid = "Anakin";
const char* password = "Skywalker31";

AsyncWebServer server(80);
WebSocketsServer webSocket = WebSocketsServer(81);

struct SensorData {
  String id;
```

```
  String bpm;
  String accX;
  String accY;
  String accZ;
  String state;
};


std::vector<SensorData> sensorDataList;

const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <title>Motion Status Dashboard</title>
  <style>
    :root {
      --bg-color: #f0f0f0;
      --text-color: #333;
      --card-bg: #ffffff;
      --topbar-bg: #333;
      --topbar-text: #fff;
    }
    body.dark-mode {
      --bg-color: #121212;
      --text-color: #f0f0f0;
      --card-bg: #1e1e1e;
      --topbar-bg: #1c1c1c;
      --topbar-text: #f0f0f0;
    }
    body {
      font-family: 'Segoe UI', Arial, sans-serif;
      background-color: var(--bg-color);
      color: var(--text-color);
```

```
  margin: 0;
  padding: 0;
  transition: all 0.3s ease;
}
.top-bar {
  background-color: var(--topbar-bg);
  color: var(--topbar-text);
  padding: 10px 20px;
  text-align: center;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.top-bar ul {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
}
.top-bar li {
  margin: 0 10px;
}
.top-bar a {
  color: var(--topbar-text);
  text-decoration: none;
  font-size: 18px;
  padding: 10px 20px;
  display: inline-block;
}
.top-bar a.active {
  background-color: #555;
  border-radius: 5px;
}
.theme-toggle {
```

```
  background: #888;
  color: #fff;
  border: none;
  padding: 8px 14px;
  border-radius: 5px;
  cursor: pointer;
  font-size: 14px;
}
.container {
  max-width: 800px;
  margin: 20px auto;
  background-color: var(--card-bg);
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 0 10px rgba(0,0,0,0.1);
}
h1 {
  text-align: center;
  font-size: 26px;
  margin-bottom: 20px;
}
.node {
  margin-bottom: 10px;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 4px;
  background-color: var(--card-bg);
  transition: background-color 0.3s ease;
}
.node h2 {
  margin-top: 0;
  font-size: 20px;
  color: var(--text-color);
}
```

```
    .pressed {
      color: green;
    }
    .released {
      color: red;
    }
  </style>
</head>
<body>
  <div class="top-bar">
    <ul>
      <li><a href="/" class="tab-link active" data-tab="motion">Motion Detection</a></li>
      <li><a href="/watch" class="tab-link" data-tab="sentry">Sentry Monitoring</a></li>
    </ul>
    <button class="theme-toggle" onclick="toggleTheme()">Toggle Theme</button>
  </div>
  <div class="container">
    <h1>Motion Status Dashboard</h1>
    <div id="statuses"></div>
  </div>

  <script>
    function toggleTheme() {
      document.body.classList.toggle('dark-mode');
    }

    var socket = new WebSocket('ws://' + window.location.hostname + ':81/');

    socket.onmessage = function(event) {
      var statusesDiv = document.getElementById('statuses');
      var message = event.data.split(":");
      var nodeId = message[0];
      var status = message[1] === "pressed" ? "No Motion" : "Motion Detected";
      var className = message[1] === "pressed" ? "released" : "pressed";
```

```
      var nodeDiv = document.getElementById(nodeId);
      if (!nodeDiv) {
        nodeDiv = document.createElement('div');
        nodeDiv.id = nodeId;
        nodeDiv.className = 'node';
        nodeDiv.innerHTML = `<h2>${nodeId}</h2><p
class="${className}">${status}</p>`;
        statusesDiv.appendChild(nodeDiv);
      } else {
        nodeDiv.querySelector('p').className = className;
        nodeDiv.querySelector('p').innerHTML = status;
      }
    };
  </script>
</body>
</html>
)rawliteral";

const char page[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <title>Sentry Monitoring Dashboard</title>
  <style>
    :root {
      --bg-color: #f0f0f0;
      --text-color: #333;
      --card-bg: #ffffff;
      --topbar-bg: #333;
      --topbar-text: #fff;
    }
    body.dark-mode {
      --bg-color: #121212;
      --text-color: #f0f0f0;
```

```
  --card-bg: #1e1e1e;
  --topbar-bg: #1c1c1c;
  --topbar-text: #f0f0f0;
}
body {
  font-family: 'Segoe UI', Arial, sans-serif;
  background-color: var(--bg-color);
  color: var(--text-color);
  margin: 0;
  padding: 0;
  transition: all 0.3s ease;
}
.top-bar {
  background-color: var(--topbar-bg);
  color: var(--topbar-text);
  padding: 10px 20px;
  text-align: center;
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.top-bar ul {
  list-style: none;
  margin: 0;
  padding: 0;
  display: flex;
}
.top-bar li {
  margin: 0 10px;
}
.top-bar a {
  color: var(--topbar-text);
  text-decoration: none;
  font-size: 18px;
```

```
  padding: 10px 20px;
  display: inline-block;
}
.top-bar a.active {
  background-color: #555;
  border-radius: 5px;
}
.theme-toggle {
  background: #888;
  color: #fff;
  border: none;
  padding: 8px 14px;
  border-radius: 5px;
  cursor: pointer;
  font-size: 14px;
}
.container {
  max-width: 800px;
  margin: 20px auto;
  background-color: var(--card-bg);
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 0 10px rgba(0,0,0,0.1);
}
h1 {
  text-align: center;
  font-size: 26px;
  margin-bottom: 20px;
}
.node {
  margin-bottom: 10px;
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 4px;
```

```
      background-color: var(--card-bg);
      transition: background-color 0.3s ease;
    }
    .node h2 {
      margin-top: 0;
      font-size: 20px;
      color: var(--text-color);
    }
    .sensor-data {
      color: var(--text-color);
    }
  </style>
</head>
<body>
  <div class="top-bar">
    <ul>
      <li><a href="/" class="tab-link" data-tab="motion">Motion Detection</a></li>
      <li><a href="/watch" class="tab-link active" data-tab="sentry">Sentry
Monitoring</a></li>
    </ul>
    <button class="theme-toggle" onclick="toggleTheme()">Toggle Theme</button>
  </div>
  <div class="container">
    <h1>Sentry Monitoring Dashboard</h1>
    <div id="statuses"></div>
  </div>

  <script>
    function toggleTheme() {
      document.body.classList.toggle('dark-mode');
    }

    function updateData(sensorId, bpm, accX, accY, accZ, state) {
  var statusesDiv = document.getElementById('statuses');
```

```
  var nodeDiv = document.getElementById(sensorId);

  const bpmVal = parseFloat(bpm);
  const x = parseFloat(accX);
  const y = parseFloat(accY);
  const z = parseFloat(accZ);

  let color = "green";
  if (state === "Fainted") color = "red";
  else if (state === "Moving") color = "brown";

  const content = `<h2>${sensorId}</h2>
                <p class="sensor-data">
                  <strong>BPM:</strong> ${isNaN(bpmVal) ? "--" : bpmVal.toFixed(1)}<br>
                  <strong>AccX:</strong> ${x.toFixed(2)},
                  <strong>AccY:</strong> ${y.toFixed(2)},
                  <strong>AccZ:</strong> ${z.toFixed(2)}<br>
                  <strong>State:</strong> <span style="color:${color}; font-
weight:bold">${state}</span>
                </p>`;

  if (!nodeDiv) {
    nodeDiv = document.createElement('div');
    nodeDiv.id = sensorId;
    nodeDiv.className = 'node';
    nodeDiv.innerHTML = content;
    statusesDiv.appendChild(nodeDiv);
  } else {
    nodeDiv.innerHTML = content;
  }
}


    function fetchData() {
```

```
      fetch('/data')
  .then(response => response.json())
  .then(data => {
    data.forEach(sensor => {
  updateData(sensor.id, sensor.bpm, sensor.accX, sensor.accY, sensor.accZ, sensor.state);
});

  })
  .catch(error => console.error('Error fetching data:', error));

    }

    setInterval(fetchData, 1000);
  </script>
</body>
</html>
)rawliteral";


void webSocketEvent(uint8_t num, WStype_t type, uint8_t* payload, size_t length) {
  if (type == WStype_TEXT) {
    Serial.printf("[%u] Received: %s\n", num, payload);
    webSocket.broadcastTXT(payload, length);
  }
}

void setup() {
  Serial.begin(115200);
  // Optional: Set Static IP (change according to your router's subnet)
IPAddress local_IP(10, 190, 20, 250);       // Desired static IP
IPAddress gateway(10, 190, 20, 1);          // Usually your router IP
IPAddress subnet(255, 255, 255, 0);         // Subnet mask

// Configure static IP
```

```
if (!WiFi.config(local_IP, gateway, subnet)) {
  Serial.println("Failed to configure Static IP");
}

WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
  delay(1000);
  Serial.println("Connecting to WiFi...");
}
Serial.println("Connected to WiFi");
Serial.print("IP Address: ");
Serial.println(WiFi.localIP());

// Start mDNS
if (MDNS.begin("dashboard")) {
  Serial.println("mDNS responder started as http://dashboard.local");
} else {
  Serial.println("Error setting up MDNS");
}

  Serial.println("Connected to WiFi");
  Serial.println(WiFi.localIP());

  if (MDNS.begin("dashboard")) {
    Serial.println("mDNS responder started: http://dashboard.local");
  } else {
    Serial.println(" Error setting up mDNS responder!");
  }

  server.on("/", HTTP_GET, [](AsyncWebServerRequest* request){
    request->send_P(200, "text/html", index_html);
  });

  server.on("/watch", HTTP_GET, [](AsyncWebServerRequest *request) {
```

```
  request->send(200, "text/html", page);
 });

 server.on("/update", HTTP_GET, [](AsyncWebServerRequest *request) {
 String sensorId, bpm, accX, accY, accZ, state;

 if (request->hasParam("sensor")) sensorId = request->getParam("sensor")->value();
 if (request->hasParam("bpm")) bpm = request->getParam("bpm")->value();
 if (request->hasParam("accX")) accX = request->getParam("accX")->value();
 if (request->hasParam("accY")) accY = request->getParam("accY")->value();
 if (request->hasParam("accZ")) accZ = request->getParam("accZ")->value();
 if (request->hasParam("state")) state = request->getParam("state")->value();

 bool updated = false;
 for (auto& data : sensorDataList) {
  if (data.id == sensorId) {
   data.bpm = bpm;
   data.accX = accX;
   data.accY = accY;
   data.accZ = accZ;
   data.state = state;  // NEW
   updated = true;
   break;
  }
 }

 if (!updated) {
  SensorData newData = { sensorId, bpm, accX, accY, accZ, state };
  sensorDataList.push_back(newData);
 }

 request->send(200, "text/plain", "Data updated");
});
```

```
  server.on("/data", HTTP_GET, [](AsyncWebServerRequest *request) {
    String json = "[";
for (size_t i = 0; i < sensorDataList.size(); ++i) {
  json += "{\"id\":\"" + sensorDataList[i].id +
          "\",\"bpm\":\"" + sensorDataList[i].bpm +
          "\",\"accX\":\"" + sensorDataList[i].accX +
          "\",\"accY\":\"" + sensorDataList[i].accY +
          "\",\"accZ\":\"" + sensorDataList[i].accZ +
          "\",\"state\":\"" + sensorDataList[i].state + "\"}";
  if (i < sensorDataList.size() - 1) json += ",";
}
json += "]";
request->send(200, "application/json", json);
  });

  server.on("/motion", HTTP_GET, [](AsyncWebServerRequest *request){
    if (request->hasParam("node") && request->hasParam("status")) {
        String node = request->getParam("node")->value();
        String status = request->getParam("status")->value();

        String message = node + ":" + (status == "motion" ? "motion" : "pressed");
        webSocket.broadcastTXT(message);
        request->send(200, "text/plain", "Motion status updated");
    } else {
        request->send(400, "text/plain", "Missing parameters");
    }
  });

  server.begin();
  webSocket.begin();
  webSocket.onEvent(webSocketEvent);
}
```

```
void loop() {
  webSocket.loop();
}
```

PLAGIARISM REPORT

Avi_report

ORIGINALITY REPORT

7%	4%	3%	5%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Thapar University, Patiala Student Paper	4%
2	www.preprints.org Internet Source	1%
3	dspace.bracu.ac.bd:8080 Internet Source	<1%
4	Jiayue Cui, Guanru Wen, Pengfei Dai, Zhicheng Liu, Xudong Lu, Long Zhao. "Transmission Line Structural Health Inspection System", 2023 Power Electronics and Power System Conference (PEPSC), 2023 Publication	<1%
5	www.jetir.org Internet Source	<1%
6	M. S. Vijaya Kumar, K. Srujan Raju, K. Rajakumar, S. Saravanakumar. "A Study on Next-Generation Materials and Devices - International Conference on A Study on Next-Generation Materials and Devices", CRC Press, 2025 Publication	<1%
7	Submitted to University of Central Lancashire Student Paper	<1%